Assemblies Page Update Outline

Assemblies Page Update Outline

Update Title

Runs Page Rename + Assemblies Management Expansion

New Official Page Name

Assemblies

Update Intent

Convert the current **Runs** page from a pipeline-run monitor into a broader **Assemblies control and management page** that helps users track, organize, evaluate, and operate assemblies as active system entities rather than isolated execution records.

Core Update Goals

- Rename the page from **Runs** to **Assemblies**
- Align labels, language, and actions with the actual assembly data model
- Expand the page from run monitoring into assembly management
- Introduce grouping and relationship concepts such as **Families**
- Add lifecycle and usage visibility so assemblies can be managed after pipeline completion
- Make the page more useful for prioritization, ownership, ecosystem understanding, and cleanup

---

1. Rename and Identity Realignment

1.1 Navigation + Page Identity

Change the page label everywhere from **Runs** to **Assemblies**.

1.2 Header Rewrite

Replace the current header and supporting copy:

- **Runs**
- “monitor, filter, and manage all pipeline runs”
- **New Run**

With:

- **Assemblies**
- “Track, organize, and operate all assemblies across the system”
- **New Assembly**

1.3 Terminology Cleanup

Remove run-centric wording where the actual entity being managed is the assembly.

Examples:

- “No runs yet” → “No assemblies yet”
- “start a new run” → “create a new assembly”
- “pipeline runs” → “assemblies and pipeline activity” where appropriate
  runs

---

2. Page Role Shift

2.1 Current Role

The page currently acts primarily as:

- a filterable status list
- a live run monitor
- a pipeline visibility table
- an entry point into individual assembly detail views

2.2 New Role

The page should become:
the operational registry for all assemblies in the Axion system

This means the page should support:

- active monitoring
- lifecycle tracking
- family grouping
- ecosystem awareness

- ownership
- usage visibility
- cleanup and deprecation decisions

---

3. Information Architecture Expansion

3.1 Keep Existing Operational Foundations

Retain and refine the parts that already work:

- status filtering
- pipeline rail visibility
- elapsed and duration context
- updated time
- row click into assembly detail
- live refresh for active assemblies

3.2 Add New Assembly Management Layers

Add new user-value layers on top of the existing table foundation:

- family grouping
- lifecycle state
- API/live usage
- ownership
- dependency awareness
- saved views
- health rollups

---

4. Assembly Families

4.1 Purpose

Introduce **Families** as a first-class organizational system for assemblies that belong to the same larger product, system, client deployment, environment, or operational cluster.

4.2 User Value

Families let users:

- understand which assemblies belong together
- see larger system groupings
- manage ecosystems instead of isolated entries
- evaluate family-wide health and risk

4.3 Proposed Family Fields

- `familyId`
- `familyName`
- `familyType`
- `familyHealth`
- `assemblyRoleInFamily`

4.4 Family Types

Examples:

- product
- client
- internal system
- service cluster
- environment
- workspace group

4.5 UI Additions

- Family column in the table
- filter by family
- group by family
- family summary card
- family quick view / drill-in panel

---

5. Ecosystem and Relationship Awareness

5.1 Purpose

Allow assemblies to be understood as connected entities in a larger operational network.

5.2 Relationship Concepts

Each assembly should be able to show:

- parent assembly
- child assemblies
- related assemblies
- upstream services
- downstream dependents
- shared registries
- shared APIs
- shared datasets

5.3 User Value

This lets the page answer:

- what depends on this assembly?
- what system is this part of?
- what other assemblies are affected by changes here?
- is this a core piece or a peripheral piece?

5.4 UI Direction

Add a relationship section through:

- expandable row details
- right-side detail drawer
- future topology mode

---

6. Lifecycle State Layer

6.1 Problem

The current page is status-heavy, but those statuses are mostly execution states such as running, completed, failed, and queued.

6.2 Solution

Add a second state system for long-term operational meaning.

6.3 Execution Status

Keep:

- queued

- running
- completed
- failed

6.4 Lifecycle State

Add:

- draft
- active
- in use
- degraded
- deprecated
- archived

6.5 User Value

This allows users to distinguish:

- assembly pipeline completion from
- assembly operational relevance

An assembly can be complete but still live, critical, stale, or ready for retirement.

---

7. API and Live Usage Tracking

7.1 Purpose

If an assembly exposes or consumes an API, the page should show whether it is actually active in the system.

7.2 Suggested Usage Signals

- requests per minute
- requests in last 24h
- active consumers
- last activity time
- error rate
- latency
- environment usage
- dependency count

7.3 User Value

This helps users identify:

- assemblies that are actively used
- assemblies that are live but degraded
- assemblies that are complete but dormant
- assemblies that should not be deprecated yet
- assemblies that are good cleanup candidates

7.4 UI Additions

- Usage column
- live / warm / idle / dormant badge
- quick usage sparkline later
- usage section in row detail or drawer

---

8. Ownership and Responsibility

8.1 Purpose

Every assembly should show who owns it and who is responsible for its operation.

8.2 Suggested Fields

- owner
- team
- control plane
- assigned agent(s)
- last operator
- approval state

8.3 User Value

This lets users quickly answer:

- who owns this?
- who changed this last?
- who should handle issues?
- what control structure governs it?

8.4 UI Additions

- Owner column
- agent/control plane indicators
- unowned filter
- responsibility block in quick detail view

---

9. Overview and Health Rollups

9.1 Purpose

The page should give immediate system awareness before the user scrolls the table.

9.2 Top Summary Metrics

Add overview cards such as:

- Total Assemblies
- Active Assemblies
- In-Use Assemblies
- Running Pipelines
- Families
- Families at Risk
- Deprecated Candidates
- Unowned Assemblies

9.3 User Value

This moves the page from passive listing to active operational command.

---

10. Saved Views and Attention States

10.1 Purpose

Users should be able to quickly jump to what needs action.

10.2 Suggested Saved Views

- Running
- Failed
- Stalled

- Slow
- Unowned
- No Family
- In Use
- Deprecated but Active
- Idle 30+ Days
- At Risk Families

10.3 User Value

This turns the page into a practical triage and management surface.

---

11. Table Evolution

11.1 Current Core Columns

The current table includes:

- Project
- Status
- Pipeline
- Preset
- Elapsed
- Duration
- Updated
- Actions

11.2 Proposed Next-State Columns

Recommended evolved table:

- Assembly
- Family
- Status
- Lifecycle
- Usage
- Pipeline
- Owner
- Updated
- Actions

11.3 Optional Future Columns

- Type
- Role
- Environment
- Dependents
- Risk level

---

12. Interaction Model

12.1 Row Click

Keep row click navigation into assembly detail.

runs

12.2 Add Quick Detail Layer

Before going fully into the assembly page, allow fast inspection through:

- hover summary
  or
- side drawer
  or
- expandable row

12.3 Quick Detail Content

Include:

- summary
- family
- lifecycle
- usage
- dependencies
- owner
- recent activity
- key actions

---

13. Empty State Rewrite

13.1 Current Problem

The empty state still uses run language such as:

- “No runs yet”
- “Launch your first run”
- “Try changing the filter or start a new run”
  runs

13.2 New Empty State Direction

Rewrite empty states to support assembly language and future organization features.

Examples:

- “No assemblies yet”
- “Create your first assembly to start building your system registry”
- “No assemblies match this view”
- “Try another filter, family, or lifecycle state”

---

14. Data Model Expansion Requirements

14.1 Existing Base

The current page already uses `Assembly[]`, status, preset, project name, idea, stage data, timestamps, and delete/navigation behavior.

14.2 New Fields Needed

To support the update, likely additions include:

- family metadata
- lifecycle state
- owner metadata
- usage metadata
- dependency metadata
- assembly type / role
- ecosystem links
- tags / labels

14.3 Backend Considerations

The assemblies API should eventually support:

- family-aware filtering
- lifecycle filtering
- usage rollups
- owner filtering
- grouping support
- optional relationship metadata

---

15. Rollout Plan

Phase 1 — Rename and Language Alignment

- rename nav label
- rename page title
- rename buttons and empty states
- update copy to assembly language

Phase 2 — Table and Filter Expansion

- add Family
- add Lifecycle
- add Usage
- add Owner
- add new saved views

Phase 3 — Overview and Management Layer

- add top summary cards
- add attention states
- add better quick filtering

Phase 4 — Relationship Intelligence

- add dependency and ecosystem metadata
- add family drill-in
- add quick detail drawer
- prepare for topology mode

Phase 5 — Advanced Operational View

- family health rollups

- API usage telemetry
- deprecation and cleanup indicators
- ecosystem visualization mode

---

16. Official Outcome Statement

After this update, the page should no longer feel like a simple historical log of runs. It should function as the **Assemblies command surface** for Axion, where users can monitor active work, organize assemblies into families, understand ecosystem relationships, track real usage, and manage lifecycle decisions from one place.

Assemblies Page Build Spec

Assemblies Page Build Spec

Spec ID

ASM-PAGE-01

Spec Title

Runs Page Rename + Assemblies Management Expansion

Spec Status

Proposed

Primary Objective

Transform the current **Runs** page into **Assemblies**, a more useful operational management surface for assemblies as living system entities, not just pipeline execution records.

Source Baseline

The current page already provides:

- assembly query from `/api/assemblies`
- active polling every 5 seconds
- filter chips for all/running/completed/failed/queued
- pipeline stage rail
- row click into `/assembly/:id`
- delete action
- run-centric header, CTA, and empty-state wording

---

1. Scope

1.1 In Scope

- rename page identity from **Runs** to **Assemblies**
- rewrite page copy to assembly-centered language
- expand list view into assembly management view
- add family grouping and filtering
- add lifecycle state layer
- add API/live usage layer
- add ownership metadata
- add top-level operational summary cards
- add saved views for actionable states
- preserve current pipeline visibility where useful

1.2 Out of Scope for Initial Pass

- full topology graph view
- advanced time-series analytics dashboards
- deep API observability explorer
- cross-page global dependency graph
- heavy backend telemetry warehouse

---

2. Page Role Definition

2.1 Current Role

The current page behaves as a run monitor and status table, with header text explicitly saying it is for monitoring and managing pipeline runs.

runs

2.2 New Role

The page will become:

Assemblies = the operational registry and command surface for system assemblies

This page should help users:

- find assemblies
- understand what each assembly belongs to
- know whether it is active or stale

- see usage and lifecycle state
- identify ownership
- spot risk and take action

---

3. Rename Specification

3.1 Page Name

Change:

- **Runs → Assemblies**
  runs

3.2 Header Copy

Replace:

- “monitor, filter, and manage all pipeline runs”
  runs

With:

- **Track, organize, and operate all assemblies across the system**

3.3 Primary CTA

Change:

- **New Run → New Assembly**

3.4 Empty State Copy

Replace:

- “No runs yet”
- “No [status] runs”
- “Launch your first run to get started.”
- “Try changing the filter or start a new run.”
  runs

With:

- “No assemblies yet”
- “No [status] assemblies”
- “Create your first assembly to get started.”
- “Try another filter, family, or lifecycle state.”

---

4. UX Goals

4.1 User Utility Goals

The update must make the page more useful for:

- operational visibility
- ecosystem organization
- ownership and accountability
- deprecation and cleanup decisions
- portfolio-level management of assemblies

4.2 Usability Outcome

A user should be able to answer these questions quickly:

- what assemblies exist?
- which ones are active?
- which family does this belong to?
- is it actually in use?
- who owns it?
- is it healthy, stale, or risky?
- what needs attention right now?

---

5. Information Architecture

5.1 Page Sections

The page should be reorganized into these layers:

A. Header

Contains:

- page title
- subtitle
- search
- create assembly CTA
- saved views selector

B. Overview Strip

Contains summary cards:

- total assemblies
- active
- in use
- failed
- unowned
- families
- families at risk

C. Filter and View Bar

Contains:

- status filters
- lifecycle filters
- family filter
- usage filter
- owner filter
- sort control
- group-by control

D. Main Table / Index View

Primary operational list of assemblies.

E. Quick Detail Surface

Right drawer, expandable row, or side panel for fast assembly inspection.

---

6. Data Model Expansion

6.1 Existing Data Already Present

The page already uses assembly data including:

- `id`
- `projectName`
- `idea`
- `status`
- `preset`
- latest stage data
- timestamps
- deletion and detail navigation behavior

6.2 New Data Requirements

Add or derive the following fields for the page model:

Identity

- `assemblyId`
- `assemblyName`
- `assemblyType`
- `assemblyRole`

Family

- `familyId`
- `familyName`
- `familyType`
- `familyHealth`
- `assemblyRoleInFamily`

Lifecycle

- `lifecycleState`
- `lifecycleUpdatedAt`

Usage

- `usageState`
- `requestsLast24h`
- `lastActivityAt`
- `activeConsumers`
- `errorRate`
- `latencyP95`

Ownership

- `ownerId`
- `ownerName`
- `teamName`
- `controlPlane`
- `assignedAgents`

Relationships

- `parentAssemblyId`
- `childAssemblyIds`
- `upstreamDependencies`
- `downstreamDependents`

Risk / Attention

- `riskLevel`
- `attentionFlags`

---

7. Table Specification

7.1 Current Table Baseline

The current table includes:

- Project
- Status
- Pipeline
- Preset
- Elapsed

- Duration
- Updated
- Actions

7.2 New Table Columns

Recommended default columns:

- **Assembly**
- **Family**
- **Status**
- **Lifecycle**
- **Usage**
- **Pipeline**
- **Owner**
- **Updated**
- **Actions**

7.3 Column Definitions

Assembly

Shows:

- `projectName` as primary title
- `idea` or short description as secondary line
  This already exists and should remain the main identity anchor.
  runs

Family

Shows:

- family name
- family type badge
- optional family health indicator

Status

Keeps current execution status behavior:

- running
- completed

- failed
- queued
  and preserves stall/slow warnings for active work. The current page already promotes stalled or slow run conditions in this area.
  runs

Lifecycle

New field:

- draft
- active
- in use
- degraded
- deprecated
- archived

Usage

Compact display such as:

- Live
- Warm
- Idle
- Dormant
  or numerical display:
- `214 req/h`
- `No traffic`
- `Last hit 2d ago`

Pipeline

Keep the current stage rail because it provides real operational value.

runs

Owner

Shows:

- owner name
- team
- control plane or assigned agent iconography

Updated

Keep updated timestamp formatting behavior. Current page already renders relative time here.

runs

Actions

Retain detail navigation and delete, then add future actions carefully.

---

8. Filter Specification

8.1 Existing Filters

Current filters are:

- all
- running
- completed
- failed
- queued

8.2 Expanded Filters

Add:

Status Filter

Keep current execution status chips.

Lifecycle Filter

Add:

- all lifecycle
- active
- in use
- degraded
- deprecated
- archived

Family Filter

Add:

- all families
- family selector
- unassigned family

Usage Filter

Add:

- live
- warm
- idle
- dormant
- no telemetry

Ownership Filter

Add:

- all owners
- mine
- unowned
- by control plane
- by team

Risk Filter

Add:

- at risk
- stalled
- failing
- deprecated but active
- family risk

---

9. Saved Views

9.1 Required Saved Views

Implement a view selector with:

- All Assemblies
- Active
- Failed
- Stalled
- In Use
- Unowned
- No Family
- Deprecated Candidates
- At Risk Families

9.2 User Value

This gives users an immediate triage workflow instead of manually stacking filters.

---

10. Family System Specification

10.1 Purpose

Families provide a first-class grouping model for assemblies that belong to one product, client system, deployment ecosystem, or operational cluster.

10.2 Functional Requirements

Users must be able to:

- assign an assembly to a family
- filter by family
- sort by family
- group the table by family
- identify family health
- identify assemblies without a family

10.3 Family Group Mode

Add optional grouped rendering:

- family header row

- family counts
- family health summary
- expandable family body

---

11. Lifecycle Specification

11.1 Dual-State Requirement

The page must support both:

- **execution status**
- **lifecycle status**

Execution status comes from pipeline activity.
Lifecycle status reflects long-term system meaning.

11.2 Lifecycle Definitions

- **draft**: created, not yet active
- **active**: usable and supported
- **in use**: actively consumed
- **degraded**: functioning with issues
- **deprecated**: replacement or retirement underway
- **archived**: retained only for record/history

11.3 Display Rules

Lifecycle should be visually separate from execution status so the user does not confuse “completed” with “healthy and still important.”

---

12. Usage Tracking Specification

12.1 Purpose

Usage metrics make the page useful after pipeline execution is over.

12.2 Minimum Usage Fields

- usage state
- requests last 24h
- active consumers
- last activity timestamp

12.3 Optional Enhanced Fields

- error rate
- latency
- environment split
- top consuming service

12.4 Display Model

Use compact badges in the table and richer metrics in the quick detail panel.

---

13. Ownership Specification

13.1 Required Ownership Fields

- primary owner
- team
- control plane
- assigned agents

13.2 Required UX Behaviors

Users must be able to:

- filter unowned assemblies
- sort by owner
- see responsibility without opening detail page

---

14. Overview Cards Specification

14.1 Required Cards

Add top summary cards for:

- Total Assemblies
- Running
- In Use
- Failed
- Unowned
- Families
- At Risk Families
- Deprecated Candidates

14.2 Behavior

Cards should be clickable and act as view shortcuts.

---

15. Quick Detail Specification

15.1 Purpose

Let users inspect an assembly without leaving the page.

15.2 Display Options

Use one of:

- right-side drawer
- expandable row
- hover card plus drawer

15.3 Quick Detail Contents

Include:

- assembly summary
- family
- execution status
- lifecycle
- usage
- owner/team/control plane
- relationship summary
- recent activity
- key actions

15.4 Navigation

Preserve row click into full assembly detail page, since current behavior already routes to `/assembly/:id`.

runs

---

16. Visual Design Direction

16.1 Keep

Retain the current polished system style:

- glass panel treatment
- chips
- stage rail
- compact operational table
- status indicators

16.2 Improve

Evolve the page into a stronger command-surface feel by adding:

- richer overview cards
- clearer grouping
- stronger status hierarchy
- better use of secondary metadata
- fewer purely empty areas

16.3 Design Priority

This page should feel like:

- a control registry
- an ecosystem dashboard
- an operations board
  not just a historical list

---

17. Backend / API Requirements

17.1 Existing Endpoint

The page already depends on:

- `GET /api/assemblies`
- `DELETE /api/assemblies/:id`
  runs

17.2 Required Expansion

The assemblies API should support enriched payloads for:

- family metadata
- lifecycle metadata
- ownership metadata
- usage rollups
- risk flags
- relationship metadata

17.3 Filtering Support

Support should exist either:

- client-side for small sets
  or
- server-side for larger datasets

Preferred server-side support for:

- family
- lifecycle
- usage state
- owner
- risk
- sorting/grouping

---

18. Migration Plan

Phase 1 — Identity Alignment

- rename Runs → Assemblies
- change CTA and empty states
- preserve current table behavior
- keep current filters and stage rail

Phase 2 — Structural Expansion

- add overview cards
- add Family, Lifecycle, Usage, Owner columns
- expand filter system
- add saved views

Phase 3 — Operational Intelligence

- add family grouping
- add quick detail drawer
- add ownership and relationship metadata
- add risk flags

Phase 4 — Advanced Ecosystem View

- add family health rollups
- add dependency awareness
- add API usage depth
- prepare topology mode

---

19. Acceptance Criteria

19.1 Rename Acceptance

- page title says **Assemblies**
- CTA says **New Assembly**
- empty states use assembly language
- no visible run-centric naming remains in this page surface

19.2 Functional Acceptance

- user can filter assemblies by more than execution status
- user can identify family membership
- user can identify lifecycle state
- user can identify whether an assembly is in use
- user can identify owner or unowned status
- user can move from summary to detail quickly

19.3 UX Acceptance

- page feels useful even when nothing is actively running
- completed assemblies still provide management value
- users can identify what needs attention without opening many rows

---

20. Priority Order

P1

- rename page to Assemblies
- copy cleanup
- CTA cleanup
- empty-state cleanup

P2

- overview cards

- Family column
- Lifecycle column
- Usage column
- Owner column

P3

- saved views
- family grouping
- unowned / no-family / at-risk filters

P4

- quick detail drawer
- relationship metadata
- family health rollups

---

21. Final Product Definition

After this update, the page should no longer be interpreted as a run log. It should become the **Assemblies operational index**: a place where users can monitor active pipeline work, manage assembly portfolios, organize assemblies into families, understand ecosystem relationships, and make lifecycle decisions from one command surface.

Assemblies Page Implementation Pass

Assemblies Page Implementation Pass

1. Route Changes

1.1 Page Identity

Keep the actual page entity tied to assemblies, but change the surfaced route identity from **Runs** to **Assemblies**.

1.2 Required Route Updates

- nav label: `Runs` → `Assemblies`
- page title reference: `RunsPage` → `AssembliesPage`
- any route metadata or breadcrumb labels using “Runs” → `Assemblies`

1.3 Route Path Recommendation

If the route is currently something like `/runs`, move toward:

- **preferred:** `/assemblies`
- keep backward compatibility redirect from `/runs` → `/assemblies` during transition

1.4 Detail Route

Keep existing assembly detail route:

- `/assembly/:id`
  This already matches the object model and should remain in place.
  runs

---

2. UI Changes

2.1 Header Block

Current header:

- title: `Runs`

- subtitle: “monitor, filter, and manage all pipeline runs”
- CTA: `New Run`
  runs

Replace with

- title: **Assemblies**
- subtitle: **Track, organize, and operate all assemblies across the system**
- CTA: **New Assembly**

Implementation Notes

This is a direct first-pass UI rename and should happen before any structural feature work.

2.2 Filter Bar

Current page uses one filter layer:

- all
- running
- completed
- failed
- queued

Upgrade To

Keep execution filter chips, then add a secondary filter row or compact toolbar for:

- lifecycle
- family
- usage
- owner
- risk

First implementation order

1. keep current chips
2. add dropdowns, not more chips, for new filter types
3. avoid overloading the top row

2.3 Overview Cards

Add a card strip above the table.

First card set

- Total Assemblies
- Running
- Failed
- In Use
- Unowned
- Families

Purpose

This makes the page useful even when nothing is actively running, which the current run-monitor framing does not do well.

2.4 Table Redesign

Current table includes:

- Project
- Status
- Pipeline
- Preset
- Elapsed
- Duration
- Updated
- Actions

New table target

- Assembly
- Family
- Status
- Lifecycle
- Usage
- Pipeline
- Owner
- Updated
- Actions

Column migration logic

- `Project` becomes **Assembly**
- `Preset` can be demoted into secondary metadata inside the Assembly cell or moved into quick detail
- `Elapsed` and `Duration` remain useful for active rows, but should no longer dominate the table for all rows
- `Updated` stays
- `Pipeline` stays because the stage rail is one of the strongest existing UI assets
  runs

2.5 Empty State Rewrite

Current empty state still says:

- “No runs yet”
- “Launch your first run to get started”
  runs

Replace with

- “No assemblies yet”
- “Create your first assembly to get started”
- filtered state: “No assemblies match this view”

2.6 Row Actions

Current actions:

- open workbench
- delete assembly
  runs

Keep

- open detail
- delete

Add later

- assign family
- change lifecycle
- mark deprecated
- open quick detail
- pin / favorite

---

3. Data Schema Additions

The current page works from `Assembly[]`, but to support the new model it needs an enriched page shape on top of the existing assembly object.

3.1 Add Family Fields

- `familyId`
- `familyName`
- `familyType`
- `familyHealth`
- `assemblyRoleInFamily`

3.2 Add Lifecycle Fields

- `lifecycleState`
- `lifecycleUpdatedAt`

3.3 Add Usage Fields

- `usageState`
- `requestsLast24h`
- `lastActivityAt`
- `activeConsumers`

3.4 Add Ownership Fields

- `ownerName`
- `teamName`
- `controlPlane`
- `assignedAgents`

3.5 Add Relationship Fields

- `parentAssemblyId`
- `childAssemblyIds`
- `upstreamDependencies`
- `downstreamDependents`

3.6 Add Risk Fields

- `riskLevel`
- `attentionFlags`

---

4. Filter Model

4.1 Execution Status Filter

Keep current filter model exactly as the first filter tier:

- all
- running
- completed
- failed
- queued

4.2 Lifecycle Filter

Add:

- all lifecycle
- draft
- active
- in use
- degraded
- deprecated
- archived

4.3 Family Filter

Add:

- all families
- unassigned
- family selector

4.4 Usage Filter

Add:

- live
- warm
- idle
- dormant
- no telemetry

4.5 Ownership Filter

Add:

- all owners
- unowned
- by team
- by control plane

4.6 Risk Filter

Add:

- all risk
- at risk
- stalled
- deprecated but active
- family risk

---

5. Table Model

5.1 Assembly Cell

Use the current `projectName` as the primary label and `idea` as the supporting line, since that pattern already exists in the page.

runs

Assembly cell contents

- assembly name
- short description
- optional type / preset tag

5.2 Family Cell

Shows:

- family name
- family type badge
- health dot

5.3 Status Cell

Keep current operational status chip behavior, including stalled/slow overlays derived from pipeline status. That is already one of the stronger parts of the page.

runs

5.4 Lifecycle Cell

Add a second badge independent from execution status.

Example:

- Status = completed
- Lifecycle = in use

That distinction is critical.

5.5 Usage Cell

Compact display:

- Live
- Warm

- Idle
- Dormant

Optional secondary text:

- `Last hit 2h ago`

5.6 Pipeline Cell

Keep `StageRail` for medium+ screens. It already provides strong at-a-glance context.

runs

5.7 Owner Cell

Shows:

- owner name
- team
- control plane badge or icon

5.8 Updated Cell

Keep current relative date formatting behavior.

runs

---

6. Quick Detail Drawer

6.1 Purpose

Do not force users to leave the page for every inspection.

6.2 Contents

- assembly summary
- family
- lifecycle
- usage

- owner
- control plane
- related assemblies
- recent activity
- actions

6.3 Interaction

Primary row click can still open `/assembly/:id` as it does today, but a dedicated icon or chevron can open quick detail first.

runs

---

7. Phased Rollout Checklist

Phase 1 — Rename Pass

- rename page label to Assemblies
- rename title
- rename subtitle
- rename CTA
- rename empty states

Phase 2 — Table Expansion

- add Family column
- add Lifecycle column
- add Usage column
- add Owner column
- keep stage rail
- demote Preset if needed

Phase 3 — Filter Expansion

- add lifecycle filter
- add family filter
- add usage filter
- add ownership filter
- add risk filter

Phase 4 — Management Layer

- add overview cards
- add saved views
- add unowned / no-family / at-risk views

Phase 5 — Relationship Layer

- add quick detail drawer
- add family grouping mode
- add relationship metadata
- add family health summary

---

8. Recommended First Build Order

If you want the cleanest implementation path, build it in this order:

Pass 1

Rename only.
No structural changes yet.

Pass 2

Add overview cards and new columns:

- Family
- Lifecycle
- Usage
- Owner

Pass 3

Add new filters and saved views.

Pass 4

Add quick detail drawer and family grouping.

That keeps the page stable while steadily increasing usefulness.

9. Official Implementation Outcome

After these changes, this page stops being a run-first monitor and becomes an **Assemblies registry and operations page**. The current assembly data/query foundation already supports that direction; the main work is restructuring the UI model around assemblies as persistent system entities instead of temporary pipeline events.

Pass 1 — Exact UI Rewrite Block

Pass 1 — Exact UI Rewrite Block

Pass Goal

Rename the page from **Runs** to **Assemblies** and make the visible UI read like an assembly management surface instead of a run log, while preserving the current assembly query, current status chips, current stage rail, current detail navigation, and current delete flow.

---

1. Component Identity Rewrite

Change

- `RunsPage` → `AssembliesPage`

Reason

The component currently exports `RunsPage` even though it queries `/api/assemblies` and navigates to `/assembly/:id`.

runs

Pass 1 Rule

Rename only the page identity and visible labels first. Do not restructure behavior yet.

---

2. Header Rewrite

Current

- Title: `Runs`
- Subtitle: `Fleet management — monitor, filter, and manage all pipeline runs`
- CTA: `New Run`
  runs

Replace With

- Title: **Assemblies**
- Subtitle: **Track, organize, and operate all assemblies across the system**
- CTA: **New Assembly**

Exact Header Block

```
<div className="flex items-center justify-between">
 <div>
   <h1 className="text-xl font-bold text-[hsl(var(--foreground))]">Assemblies</h1>
   <p className="text-xs text-[hsl(var(--muted-foreground))] mt-1">
     Track, organize, and operate all assemblies across the system
   </p>
 </div>
 <button
   onClick={() => setLocation("/new")}
   className="flex items-center gap-2 px-4 py-2 rounded-md bg-[hsl(var(--primary))]
text-[hsl(var(--primary-foreground))] hover:opacity-90 transition font-medium text-sm"
 >
   <Plus className="w-4 h-4" />
   New Assembly
 </button>
</div>
```

---

3. Filter Strip Rewrite

Keep

Keep the current chip structure and current execution-status logic:

- all
- running
- completed
- failed
- queued
  runs

Change

Update the labels so they read more clearly in assembly context.

Recommended chip labels

- `All`
- `Running`
- `Completed`
- `Failed`
- `Queued`

The current chip label `Active` maps to `running`, but on the new Assemblies page that word will become overloaded later once lifecycle state exists. For Pass 1, make it literal.

Exact Chip Update

```
const filterChips: { key: FilterStatus; label: string; icon: typeof Activity }[] = [
 { key: "all", label: "All", icon: Activity },
 { key: "running", label: "Running", icon: Radio },
 { key: "completed", label: "Completed", icon: CheckCircle2 },
 { key: "failed", label: "Failed", icon: XCircle },
 { key: "queued", label: "Queued", icon: Clock },
];
```

---

4. Empty State Rewrite

Current

The page currently uses:

- `No runs yet`
- `No ${activeFilter} runs`
- `Launch your first run to get started.`
- `Try changing the filter or start a new run.`
- `New Run` button in empty state
  runs

Replace With

- `No assemblies yet`
- `No ${activeFilter} assemblies`
- `Create your first assembly to get started.`

- `Try changing the filter or create a new assembly.`
- `New Assembly`

Exact Empty State Block

```
<GlassPanel solid className="p-16 text-center">
 <div className="w-14 h-14 rounded-full bg-[hsl(var(--muted))] flex items-center justify-center
mx-auto mb-4">
   <Rocket className="w-7 h-7 text-[hsl(var(--muted-foreground))]" />
 </div>
 <p className="text-sm font-medium text-[hsl(var(--foreground))] mb-1">
   {activeFilter === "all"
     ? "No assemblies yet"
     : `No ${activeFilter} assemblies`}
 </p>
 <p className="text-xs text-[hsl(var(--muted-foreground))] mb-4">
   {activeFilter === "all"
     ? "Create your first assembly to get started."
     : "Try changing the filter or create a new assembly."}
 </p>
 {activeFilter === "all" && (
   <button
     onClick={() => setLocation("/new")}
     className="inline-flex items-center gap-2 px-4 py-2 rounded-md bg-[hsl(var(--primary))]
text-[hsl(var(--primary-foreground))] hover:opacity-90 transition font-medium text-sm"
   >
     <Plus className="w-4 h-4" />
     New Assembly
   </button>
 )}
</GlassPanel>
```

---

5. Table Header Rewrite

Current

The table headers are:

- Project
- Status
- Pipeline
- Preset

- Elapsed
- Duration
- Updated
- Actions
  runs

Pass 1 Change

Rename only what improves clarity immediately without requiring new data.

Replace With

- **Assembly**
- **Status**
- **Pipeline**
- **Preset**
- **Elapsed**
- **Duration**
- **Updated**
- **Actions**

Exact Header Row

```
<tr className="border-b border-[hsl(var(--border))]">
 <th className="text-left px-4 py-2.5 text-system-label">Assembly</th>
 <th className="text-left px-4 py-2.5 text-system-label">Status</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden md:table-cell">Pipeline</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden md:table-cell">Preset</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Elapsed</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Duration</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Updated</th>
 <th className="text-right px-4 py-2.5 text-system-label">Actions</th>
</tr>
```

---

6. Primary Cell Rewrite

Current

The first column renders `projectName` with `idea` beneath it. That is already the correct base pattern for the future Assemblies page.

runs

Pass 1 Change

Keep the structure, but treat it explicitly as the assembly identity cell.

Recommended micro-copy interpretation

- primary line = assembly name
- secondary line = assembly summary / idea

No logic change needed yet.

---

7. Actions Copy Cleanup

Current

The row action uses a title of `Open Workbench` while routing to `/assembly/:id`.

runs

Pass 1 Recommendation

Rename the action title to something assembly-centered.

Replace

- `Open Workbench`

With either

- `Open Assembly`
  or
- `Open Details`

Exact Action Update

<button

```
 onClick={() => setLocation(`/assembly/${a.id}`)}
 className="p-1.5 rounded hover:bg-[hsl(var(--accent))] transition-colors"
 title="Open Assembly"
>
 <ExternalLink className="w-3.5 h-3.5 text-[hsl(var(--muted-foreground))]" />
</button>
```

---

8. Status Presentation Rule

Keep As-Is

Do not rewrite the status logic in Pass 1.

The current page already has one of the better operational behaviors:

- running assemblies pulse
- stalled assemblies show warning/failure treatment
- stage rail remains visible
- elapsed and idle time show useful live context
  runs

That should stay untouched for the first pass.

---

9. Preset / Elapsed / Duration Rule

Keep For Pass 1

Do not remove:

- Preset
- Elapsed
- Duration

Even though the page is moving beyond run-monitor behavior, those fields are still useful and already implemented.

runs

Pass 1 Position

Keep them now.
Demote or reorganize them in Pass 2 when Family, Lifecycle, Usage, and Owner are added.

---

10. Pass 1 Optional Add-On

One small addition fits cleanly into Pass 1 without needing backend changes:

Add a tiny page-level summary strip above the table

Using current counts only:

- All
- Running
- Completed
- Failed
- Queued

This is optional because the chip counts already exist, but cards would make the page feel more intentional.

Recommended first-pass cards

- Total Assemblies
- Running
- Completed
- Failed
- Queued

This can be built from the existing `counts` object already in the page.

runs

---

11. Pass 1 Acceptance Criteria

Pass 1 is complete when:

- page title says **Assemblies**
- subtitle no longer says “pipeline runs”
- CTA says **New Assembly**
- empty states use assembly language

- table header says **Assembly** instead of **Project**
- row open action says **Open Assembly** or **Open Details**
- current assembly query, filtering, stage rail, row navigation, and delete behavior still work unchanged

---

12. Pass 1 Result

After this pass, the page will still function almost exactly the same technically, but it will stop misrepresenting itself. It will read as an assembly page first, which sets the foundation for the next pass where we add:

- Family
- Lifecycle
- Usage
- Owner

That is the right next structural jump.

Pass 2 — Exact Table Expansion Block

Pass 2 — Exact Table Expansion Block

Pass Goal

Expand the main table from:

- Project
- Status
- Pipeline
- Preset
- Elapsed
- Duration
- Updated
- Actions

into a more useful assembly-management table with room for:

- Family
- Lifecycle
- Usage
- Owner

without breaking the current strengths:

- status monitoring
- stage rail
- elapsed timing
- updated timestamp
- row click to assembly detail.

---

1. New Pass 2 Table Shape

Replace current table target with

- **Assembly**
- **Family**
- **Status**
- **Lifecycle**

- **Usage**
- **Pipeline**
- **Owner**
- **Updated**
- **Actions**

Why this is the right shift

Right now the table is still biased toward pipeline execution because `Preset`, `Elapsed`, and `Duration` occupy more space than long-term management fields. That made sense for a runs page, but not for an assemblies page.

Pass 2 changes the table so the first screen answers:

- what is this assembly?
- what family is it in?
- what state is it in?
- is it being used?
- who owns it?

That is the right priority order for the upgraded page.

---

2. Column Migration Rules

2.1 Assembly

Current

`Project` column shows:

- `projectName`
- `idea` beneath it
  runs

Pass 2

Rename and keep this as the main identity cell.

New header

`Assembly`

Cell behavior

- primary line = `projectName`
- secondary line = `idea`
- optional tertiary badge = `preset` moved here as a small metadata chip

Why

The current `projectName + idea` pattern is already correct for the assembly identity surface.

runs

2.2 Family

New column

`Family`

Temporary data strategy

Until backend family metadata exists, use placeholder or derived display:

- `Unassigned`
  or
- fallback from known grouping if one exists later

Cell behavior

Show:

- family name
- small muted family type if available later

Pass 2 rule

Even if the data is placeholder-only at first, the column should exist now because it signals the new page model clearly.

2.3 Status

Current

The page already has strong status rendering:

- active chip
- stalled / slow warning behavior
- pulse for running assemblies

Pass 2

Keep this logic intact.

Rule

Do not weaken the current operational status cell. It is one of the strongest parts of the page already.

2.4 Lifecycle

New column

`Lifecycle`

Temporary first values

Until real lifecycle data exists, use a simple derived fallback:

- `Active` for running
- `Draft` for queued
- `Available` for completed
- `Attention` for failed

Important

These are temporary placeholders only for Pass 2 surface testing.
Later they should become:

- draft
- active

- in use
- degraded
- deprecated
- archived

Why add it now

This column introduces the critical idea that execution status and lifecycle are not the same thing.

2.5 Usage

New column

`Usage`

Temporary first values

Until telemetry exists:

- running → `Live`
- completed → `Idle`
- queued → `Pending`
- failed → `Blocked`

Optional secondary text

- `No telemetry yet`

Why

Even placeholder usage states push the page toward operational usefulness and prepare the surface for real API/activity tracking later.

2.6 Pipeline

Current

The table already uses `StageRail` for pipeline visibility.

Pass 2

Keep exactly as-is.

Rule

Pipeline remains a core mid-table operational signal.

2.7 Owner

New column

`Owner`

Temporary first value

Until ownership data exists:

- `Unassigned`

Later expansion

This should eventually show:

- owner name
- team
- control plane

Why add it now

This is one of the highest-value management fields and should be visually present early.

2.8 Updated

Current

The table already shows relative update time via `formatDate(a.updatedAt)`.

Pass 2

Keep this column.

Rule

Updated time remains useful for both active and inactive assemblies.

2.9 Actions

Current

Actions include open detail and delete. The open action still says `Open Workbench`.

runs

Pass 2

Keep:

- open assembly
- delete

Rename the open action to:

- `Open Assembly`

---

3. Exact Pass 2 Header Row

```
<tr className="border-b border-[hsl(var(--border))]">
 <th className="text-left px-4 py-2.5 text-system-label">Assembly</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden md:table-cell">Family</th>
 <th className="text-left px-4 py-2.5 text-system-label">Status</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Lifecycle</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Usage</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden md:table-cell">Pipeline</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden xl:table-cell">Owner</th>
 <th className="text-left px-4 py-2.5 text-system-label hidden lg:table-cell">Updated</th>
 <th className="text-right px-4 py-2.5 text-system-label">Actions</th>
```

```
</tr>
```

---

4. Exact Pass 2 Row Model

4.1 Assembly Cell

Move preset into the Assembly cell instead of leaving it as its own column.

```
<td className="px-4 py-3">
 <div className="flex flex-col gap-1">
   <div className="font-medium text-[hsl(var(--foreground))] text-[13px]">
     {a.projectName}
   </div>

   {a.idea && (
     <div className="text-[11px] text-[hsl(var(--muted-foreground))] truncate max-w-[240px]">
       {a.idea}
     </div>
   )}

   <div className="flex items-center gap-2">
     <span className="text-[10px] font-mono-tech text-[hsl(var(--muted-foreground))]">
       {a.preset || "default"}
     </span>
   </div>
 </div>
</td>
```

4.2 Family Cell

```
<td className="px-4 py-3 hidden md:table-cell">
 <div className="flex flex-col">
   <span className="text-xs text-[hsl(var(--foreground))]">
     {(a as any).familyName || "Unassigned"}
   </span>
   <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
     {(a as any).familyType || "No family"}
   </span>
 </div>
</td>
```

4.3 Status Cell

Keep the existing status block structure because it already handles stalled/slow behavior well.

No major rewrite here in Pass 2.

4.4 Lifecycle Cell

```
<td className="px-4 py-3 hidden lg:table-cell">
 <span className="text-xs text-[hsl(var(--muted-foreground))]">
  {(a as any).lifecycleState ||
    (a.status === "running"
      ? "Active"
      : a.status === "queued"
      ? "Draft"
      : a.status === "completed"
      ? "Available"
      : "Attention")}
 </span>
</td>
```

4.5 Usage Cell

```
<td className="px-4 py-3 hidden lg:table-cell">
 <div className="flex flex-col">
   <span className="text-xs text-[hsl(var(--foreground))]">
     {(a as any).usageState ||
       (a.status === "running"
         ? "Live"
         : a.status === "completed"
         ? "Idle"
         : a.status === "queued"
         ? "Pending"
         : "Blocked")}
   </span>
   <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
     No telemetry yet
   </span>
 </div>
```

```
</td>
```

4.6 Pipeline Cell

Keep current `StageRail` usage unchanged.

runs

```
<td className="px-4 py-3 hidden md:table-cell">
 <StageRail stages={parseStagesFromAssembly((a as any).latestStages)} />
</td>
```

4.7 Owner Cell

```
<td className="px-4 py-3 hidden xl:table-cell">
 <div className="flex flex-col">
   <span className="text-xs text-[hsl(var(--foreground))]">
     {(a as any).ownerName || "Unassigned"}
   </span>
   <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
     {(a as any).controlPlane || "No control plane"}
   </span>
 </div>
</td>
```

4.8 Updated Cell

Keep current updated rendering.

```
<td className="px-4 py-3 hidden lg:table-cell">
 <span className="text-xs text-[hsl(var(--muted-foreground))]">
   {formatDate(a.updatedAt)}
 </span>
</td>
```

---

5. Columns to Remove or Demote in Pass 2

Remove as standalone columns

- Preset
- Elapsed
- Duration

These fields are still useful, but they should no longer dominate the default table layout on an Assemblies page. Right now they take up three columns in the current runs-focused table.

Where they go instead

- `Preset` moves into the Assembly identity cell
- `Elapsed` and `Duration` move into:
    - quick detail later
    - optional hover metadata
    - mobile/secondary row metadata if needed

Reason

This creates room for the higher-value long-term management fields:

- Family
- Lifecycle
- Usage
- Owner

---

6. Responsive Behavior Rules

Mobile

Show only:

- Assembly
- Status
- Actions

Medium

Add:

- Family
- Pipeline

Large

Add:

- Lifecycle
- Usage
- Updated

XL

Add:

- Owner

This keeps the table dense but usable.

---

7. Pass 2 Acceptance Criteria

Pass 2 is complete when:

- `Assembly` replaces `Project`
- `Family` column exists
- `Lifecycle` column exists
- `Usage` column exists
- `Owner` column exists
- `Preset` is no longer a standalone column
- `Elapsed` and `Duration` are no longer standalone columns
- current status logic still works
- current stage rail still works
- current updated timestamp still works
- row click to `/assembly/:id` still works.

Pass 3 — Exact Filter and Saved-Views Block

Pass 3 — Exact Filter and Saved-Views Block

Pass Goal

Expand the page from a single-layer execution-status filter into a usable control surface with:

- execution status filters
- saved views
- family filter
- lifecycle filter
- usage filter
- ownership filter
- risk filter

while preserving the current strengths:

- fast chip-based filtering
- status counts
- simple derived `filtered` data flow
- no deep behavioral disruption yet.

---

1. Current Filter Baseline

The current page has:

- `type FilterStatus = "all" | "running" | "completed" | "failed" | "queued"`
- `filterChips` with chip label/icon config
- `activeFilter`
- `counts`
- `filtered = activeFilter === "all" ? assemblies : assemblies.filter(...)` runs

That means the existing page already has a clean foundation for layered filters.

---

2. Pass 3 Filter Model Shift

Current model

One filter controls everything:

- `activeFilter`

New model

Split filtering into:

- `statusFilter`
- `savedView`
- `familyFilter`
- `lifecycleFilter`
- `usageFilter`
- `ownerFilter`
- `riskFilter`
- `searchQuery` later if needed

Why

This keeps execution status as one filter axis instead of making it the whole page state.

---

3. Exact New Types

3.1 Status Filter Type

Keep current status coverage, but rename the state variable from `activeFilter` to `statusFilter`.

type StatusFilter = "all" | "running" | "completed" | "failed" | "queued";

3.2 Saved View Type

```
type SavedView =
 | "all"
 | "active"
 | "failed"
 | "stalled"
 | "in_use"
 | "unowned"
 | "no_family"
 | "deprecated_candidates"
 | "at_risk";
```

3.3 Family Filter Type

```
type FamilyFilter = "all" | "assigned" | "unassigned";
```

3.4 Lifecycle Filter Type

```
type LifecycleFilter =
 | "all"
 | "draft"
 | "active"
 | "available"
 | "in_use"
 | "degraded"
 | "deprecated"
 | "archived";
```

3.5 Usage Filter Type

```
type UsageFilter = "all" | "live" | "warm" | "idle" | "dormant" | "pending" | "blocked";
```

3.6 Owner Filter Type

```
type OwnerFilter = "all" | "assigned" | "unassigned";
```

3.7 Risk Filter Type

```
type RiskFilter = "all" | "at_risk" | "stalled" | "slow" | "blocked";
```

---

4. Exact New State Block

Replace the current single filter state:

```
const [activeFilter, setActiveFilter] = useState<FilterStatus>("all");
```

with:

```
const [statusFilter, setStatusFilter] = useState<StatusFilter>("all");
const [savedView, setSavedView] = useState<SavedView>("all");
const [familyFilter, setFamilyFilter] = useState<FamilyFilter>("all");
const [lifecycleFilter, setLifecycleFilter] = useState<LifecycleFilter>("all");
const [usageFilter, setUsageFilter] = useState<UsageFilter>("all");
const [ownerFilter, setOwnerFilter] = useState<OwnerFilter>("all");
const [riskFilter, setRiskFilter] = useState<RiskFilter>("all");
```

This is the main architectural shift in Pass 3.

---

5. Exact Status Chip Rewrite

The current chip row is already structurally correct: it maps over `filterChips`, uses a count badge, and toggles active state in one click. Keep that interaction model.

Replace the type and labels with:

```
const statusChips: { key: StatusFilter; label: string; icon: typeof Activity }[] = [
 { key: "all", label: "All", icon: Activity },
 { key: "running", label: "Running", icon: Radio },
 { key: "completed", label: "Completed", icon: CheckCircle2 },
 { key: "failed", label: "Failed", icon: XCircle },
 { key: "queued", label: "Queued", icon: Clock },
];
```

Replace chip usage with:

```
<div className="flex items-center gap-2 flex-wrap">
 {statusChips.map((chip) => {
   const isActive = statusFilter === chip.key;
   return (
     <button
```

```
      key={chip.key}
      onClick={() => setStatusFilter(chip.key)}
      className={`inline-flex items-center gap-1.5 px-3 py-1.5 rounded-full text-xs font-medium
transition-all duration-150 ${
        isActive
          ? "bg-[hsl(var(--primary)/0.15)] text-[hsl(var(--primary))] border
border-[hsl(var(--primary)/0.3)]"
          : "bg-[hsl(var(--muted))] text-[hsl(var(--muted-foreground))] border
border-[hsl(var(--border))] hover:border-[hsl(var(--primary)/0.2)]
hover:text-[hsl(var(--foreground))]"
      }`}
    >
      <chip.icon className="w-3 h-3" />
      {chip.label}
      <span
        className={`ml-0.5 tabular-nums ${
          isActive
            ? "text-[hsl(var(--primary))]"
            : "text-[hsl(var(--muted-foreground)/0.7)]"
        }`}
      >
        {counts[chip.key]}
      </span>
    </button>
  );
})}
</div>
```

---

6. Add Saved Views Bar

Why saved views come first

Saved views are the fastest way to make the new table feel smarter before every filter has real backend data.

Exact saved views config

```
const savedViews: { key: SavedView; label: string }[] = [
 { key: "all", label: "All Assemblies" },
 { key: "active", label: "Active" },
 { key: "failed", label: "Failed" },
```

```
 { key: "stalled", label: "Stalled" },
 { key: "in_use", label: "In Use" },
 { key: "unowned", label: "Unowned" },
 { key: "no_family", label: "No Family" },
 { key: "deprecated_candidates", label: "Deprecated Candidates" },
 { key: "at_risk", label: "At Risk" },
];
```

Exact UI block

```
<div className="flex items-center gap-2 flex-wrap">
 <span className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
   Views
 </span>

 {savedViews.map((view) => {
   const isActive = savedView === view.key;
   return (
     <button
       key={view.key}
       onClick={() => setSavedView(view.key)}
       className={`px-3 py-1.5 rounded-md text-xs font-medium transition-colors ${
         isActive
           ? "bg-[hsl(var(--primary)/0.15)] text-[hsl(var(--primary))] border
border-[hsl(var(--primary)/0.25)]"
           : "bg-[hsl(var(--muted)/0.65)] text-[hsl(var(--muted-foreground))] border
border-[hsl(var(--border))] hover:text-[hsl(var(--foreground))]"
       }`}
     >
       {view.label}
     </button>
   );
 })}
</div>
```

---

7. Add Secondary Filter Toolbar

Pass 3 rule

Do not add more chips for every new filter type. Use compact selects.

Exact toolbar structure

```
<div className="flex items-center gap-2 flex-wrap">
 <div className="inline-flex items-center gap-1.5 px-2.5 py-1.5 rounded-md border
border-[hsl(var(--border))] bg-[hsl(var(--muted)/0.45)]">
   <Filter className="w-3.5 h-3.5 text-[hsl(var(--muted-foreground))]" />
   <span className="text-xs text-[hsl(var(--muted-foreground))]">Filters</span>
 </div>

 <select
   value={familyFilter}
   onChange={(e) => setFamilyFilter(e.target.value as FamilyFilter)}
   className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
 >
   <option value="all">All Families</option>
   <option value="assigned">Assigned Family</option>
   <option value="unassigned">No Family</option>
 </select>

 <select
   value={lifecycleFilter}
   onChange={(e) => setLifecycleFilter(e.target.value as LifecycleFilter)}
   className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
 >
   <option value="all">All Lifecycle</option>
   <option value="draft">Draft</option>
   <option value="active">Active</option>
   <option value="available">Available</option>
   <option value="in_use">In Use</option>
   <option value="degraded">Degraded</option>
   <option value="deprecated">Deprecated</option>
   <option value="archived">Archived</option>
 </select>

 <select
   value={usageFilter}
   onChange={(e) => setUsageFilter(e.target.value as UsageFilter)}
   className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
 >
   <option value="all">All Usage</option>
   <option value="live">Live</option>
```

```
  <option value="warm">Warm</option>
  <option value="idle">Idle</option>
  <option value="dormant">Dormant</option>
  <option value="pending">Pending</option>
  <option value="blocked">Blocked</option>
 </select>

 <select
  value={ownerFilter}
  onChange={(e) => setOwnerFilter(e.target.value as OwnerFilter)}
  className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
 >
  <option value="all">All Owners</option>
  <option value="assigned">Assigned Owner</option>
  <option value="unassigned">Unowned</option>
 </select>

 <select
  value={riskFilter}
  onChange={(e) => setRiskFilter(e.target.value as RiskFilter)}
  className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
 >
  <option value="all">All Risk</option>
  <option value="at_risk">At Risk</option>
  <option value="stalled">Stalled</option>
  <option value="slow">Slow</option>
  <option value="blocked">Blocked</option>
 </select>
</div>
```

---

8. Exact Derived Filter Logic

The current page only does one pass:

```
const filtered =
 activeFilter === "all"
  ? assemblies
  : assemblies.filter((a) => a.status === activeFilter);
```

That needs to become a layered pipeline.

Replace with:

```
const filtered = assemblies.filter((a) => {
 const statusEntry =
   a.status === "running"
     ? pipelineStatus?.activeRuns?.find((s) => s.assemblyId === a.id)
     : undefined;

 const stallLevel = statusEntry ? getStallLevel(statusEntry.stalledMs) : "none";

 const familyName = (a as any).familyName || null;
 const lifecycleState =
   (a as any).lifecycleState ||
   (a.status === "running"
     ? "active"
     : a.status === "queued"
     ? "draft"
     : a.status === "completed"
     ? "available"
     : "degraded");

 const usageState =
   (a as any).usageState ||
   (a.status === "running"
     ? "live"
     : a.status === "completed"
     ? "idle"
     : a.status === "queued"
     ? "pending"
     : "blocked");

 const ownerName = (a as any).ownerName || null;

 const matchesStatus =
   statusFilter === "all" ? true : a.status === statusFilter;

 const matchesFamily =
   familyFilter === "all"
     ? true
     : familyFilter === "assigned"
     ? !!familyName
     : !familyName;

 const matchesLifecycle =
```

```
  lifecycleFilter === "all" ? true : lifecycleState === lifecycleFilter;

const matchesUsage =
  usageFilter === "all" ? true : usageState === usageFilter;

const matchesOwner =
  ownerFilter === "all"
    ? true
    : ownerFilter === "assigned"
    ? !!ownerName
    : !ownerName;

const matchesRisk =
  riskFilter === "all"
    ? true
    : riskFilter === "stalled"
    ? stallLevel === "critical"
    : riskFilter === "slow"
    ? stallLevel === "warning"
    : riskFilter === "blocked"
    ? a.status === "failed"
    : stallLevel === "critical" || a.status === "failed";

const matchesSavedView =
  savedView === "all"
    ? true
    : savedView === "active"
    ? a.status === "running" || lifecycleState === "active"
    : savedView === "failed"
    ? a.status === "failed"
    : savedView === "stalled"
    ? stallLevel === "critical"
    : savedView === "in_use"
    ? usageState === "live" || lifecycleState === "in_use"
    : savedView === "unowned"
    ? !ownerName
    : savedView === "no_family"
    ? !familyName
    : savedView === "deprecated_candidates"
    ? usageState === "idle" && a.status === "completed"
    : stallLevel === "critical" || a.status === "failed";

return (
  matchesStatus &&
```

```
  matchesFamily &&
  matchesLifecycle &&
  matchesUsage &&
  matchesOwner &&
  matchesRisk &&
  matchesSavedView
 );
});
```

---

9. Saved View Behavior Rules

All Assemblies

Shows everything.

Active

Show assemblies that are:

- currently running
  or
- lifecycle active

Failed

Show failed assemblies only.

Stalled

Use current stall detection via `pipelineStatus` and `getStallLevel`, since the page already computes that for running rows.

runs

In Use

Show:

- `usageState === "live"`
  or
- `lifecycleState === "in_use"`

Unowned

No owner assigned.

No Family

No family assigned.

Deprecated Candidates

Use a temporary heuristic:

- completed
- idle
- no live usage

At Risk

Show:

- failed
- stalled
- blocked
- degraded later

---

10. Counts Strategy

The current `counts` object is status-only and should stay that way for the chip row because it matches the current UX pattern exactly.

Keep:

```
const counts: Record<StatusFilter, number> = {
 all: assemblies.length,
```

```
 running: assemblies.filter((a) => a.status === "running").length,
 completed: assemblies.filter((a) => a.status === "completed").length,
 failed: assemblies.filter((a) => a.status === "failed").length,
 queued: assemblies.filter((a) => a.status === "queued").length,
};
```

Do not

Do not try to compute separate per-view counts yet in Pass 3. That adds noise too early.

---

11. Filter Layout Order

Use this exact stacking order:

First row

Status chips

Second row

Saved views

Third row

Secondary filter toolbar

That gives the page a clear control hierarchy:

1. quick execution status
2. curated management views
3. fine-grained filter refinement

---

12. Clear Filters Action

Add one compact reset control.

Exact button

```
<button
 onClick={() => {
   setSavedView("all");
   setStatusFilter("all");
   setFamilyFilter("all");
   setLifecycleFilter("all");
   setUsageFilter("all");
   setOwnerFilter("all");
   setRiskFilter("all");
 }}
 className="px-3 py-1.5 rounded-md text-xs border border-[hsl(var(--border))]
text-[hsl(var(--muted-foreground))] hover:text-[hsl(var(--foreground))]"
>
 Reset Filters
</button>
```

Place it at the end of the secondary filter toolbar.

---

13. Pass 3 Acceptance Criteria

Pass 3 is complete when:

- execution status chips still work
- `activeFilter` has been replaced with `statusFilter`
- saved views exist
- family filter exists
- lifecycle filter exists
- usage filter exists
- owner filter exists
- risk filter exists
- filtering now applies as a layered pipeline
- stalled view can leverage the same stall data already used in the status cell
- current assembly list, row navigation, and status rendering still work.

---

14. Result of Pass 3

After this pass, the table becomes manageable as an assemblies page instead of just broader. That is the key difference: Pass 2 adds more meaning to each row, and Pass 3 gives the user the control surface needed to actually operate that meaning.

Pass 4 — Exact Overview Cards and Top-Summary

Pass 4 — Exact Overview Cards and Top-Summary Block

Pass Goal

Add a top summary strip above the filters and table so the page becomes useful at a glance.

This pass should answer, before the user scrolls:

- how many assemblies exist?
- how many are active right now?
- how many failed?
- how many are queued?
- are any runs stalled?
- when is the system quiet vs busy?

That is the right next step because the current page still opens with only a title and filter chips, which is thin for a page meant to become an assemblies command surface.

runs

---

1. Placement

Insert order

The page should render in this order:

1. Header
2. Overview cards
3. Status chips
4. Saved views
5. Secondary filters
6. Table / empty state

Why

The current page goes straight from header to filter chips.

runs

Adding overview cards between those layers makes the page feel like a control surface rather than just a filtered list.

---

2. First-Pass Card Set

Use a **6-card strip** in Pass 4:

- Total Assemblies
- Running
- Completed
- Failed
- Queued
- Stalled / At Risk

Why these six

Five of these are already directly supported by the current `counts` object. The sixth can be derived from current stall logic because the table already computes stalled/slow state using `pipelineStatus` and `getStallLevel`.

Do **not** use Family, Usage, Owner, or Lifecycle cards yet unless those data fields actually exist in the payload.

---

3. Exact Derived Metrics Block

Add this above the `return` block, after the existing `counts` object:

```
const activeRuns = pipelineStatus?.activeRuns ?? [];

const stalledCount = activeRuns.filter(
 (run) => getStallLevel(run.stalledMs) === "critical"
).length;
```

```
const slowCount = activeRuns.filter(
 (run) => getStallLevel(run.stalledMs) === "warning"
).length;

const completedPct =
 assemblies.length > 0
   ? Math.round((counts.completed / assemblies.length) * 100)
   : 0;

const healthyPct =
 assemblies.length > 0
   ? Math.round(((counts.completed + counts.running + counts.queued - counts.failed) /
assemblies.length) * 100)
   : 0;
```

Why this works

- `counts` already exists for total/running/completed/failed/queued.
  runs
- `pipelineStatus?.activeRuns` is already used in row-level status rendering.
- `getStallLevel` is already imported and used in the page.

So this pass is additive, not speculative.

---

4. Card Interaction Model

Each card should be clickable and act like a shortcut.

Card click mapping

- **Total Assemblies** → `setStatusFilter("all")`, `setSavedView("all")`
- **Running** → `setStatusFilter("running")`
- **Completed** → `setStatusFilter("completed")`
- **Failed** → `setStatusFilter("failed")`
- **Queued** → `setStatusFilter("queued")`
- **Stalled / At Risk** → `setSavedView("stalled")`

This makes the summary layer operational, not decorative.

5. Exact Card Grid Block

Insert this directly below the header and above the filter rows:

```
<div className="grid grid-cols-1 sm:grid-cols-2 xl:grid-cols-6 gap-3">
 <button
   onClick={() => {
     setSavedView("all");
     setStatusFilter("all");
   }}
   className="text-left"
 >
   <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
     <div className="flex items-start justify-between">
       <div>
         <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
           Total Assemblies
         </p>
         <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
           {counts.all}
         </p>
       </div>
       <Activity className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
     </div>
     <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
       Full registry footprint
     </p>
   </GlassPanel>
 </button>

 <button
   onClick={() => setStatusFilter("running")}
   className="text-left"
 >
   <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
     <div className="flex items-start justify-between">
       <div>
         <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
           Running
```

```
        </p>
        <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
          {counts.running}
        </p>
      </div>
      <Radio className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
    </div>
    <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      Active pipeline work
    </p>
  </GlassPanel>
</button>

<button
  onClick={() => setStatusFilter("completed")}
  className="text-left"
>
  <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
    <div className="flex items-start justify-between">
      <div>
        <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
          Completed
        </p>
        <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
          {counts.completed}
        </p>
      </div>
      <CheckCircle2 className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
    </div>
    <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      {completedPct}% of total assemblies
    </p>
  </GlassPanel>
</button>

<button
  onClick={() => setStatusFilter("failed")}
  className="text-left"
>
  <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
    <div className="flex items-start justify-between">
      <div>
```

```
        <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
          Failed
        </p>
        <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
          {counts.failed}
        </p>
      </div>
      <XCircle className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
    </div>
    <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      Assemblies needing follow-up
    </p>
  </GlassPanel>
</button>

<button
  onClick={() => setStatusFilter("queued")}
  className="text-left"
>
  <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
    <div className="flex items-start justify-between">
      <div>
        <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
          Queued
        </p>
        <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
          {counts.queued}
        </p>
      </div>
      <Clock className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
    </div>
    <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      Pending execution
    </p>
  </GlassPanel>
</button>

<button
  onClick={() => setSavedView("stalled")}
  className="text-left"
>
  <GlassPanel solid className="p-4 h-full hover:border-[hsl(var(--primary)/0.25)]
transition-colors">
```

```
    <div className="flex items-start justify-between">
      <div>
        <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
          Stalled / At Risk
        </p>
        <p className="mt-2 text-2xl font-semibold text-[hsl(var(--foreground))]">
          {stalledCount}
        </p>
      </div>
      <AlertTriangle className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
    </div>
    <p className="mt-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      {slowCount > 0 ? `${slowCount} additional slow` : "No slow runs detected"}
    </p>
  </GlassPanel>
 </button>
</div>
```

---

6. Optional Health Banner Row

Below the card grid, add one thin system-health strip. This is optional in Pass 4, but it fits cleanly with the current data.

Exact banner block

```
<GlassPanel solid className="px-4 py-3 flex flex-wrap items-center gap-3">
 <div className="inline-flex items-center gap-2">
   <span className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
     Registry Health
   </span>
   <span className="text-sm font-medium text-[hsl(var(--foreground))]">
     {healthyPct}%
   </span>
 </div>

 <div className="h-4 w-px bg-[hsl(var(--border))]" />

 <div className="text-xs text-[hsl(var(--muted-foreground))]">
   {counts.running > 0
     ? `${counts.running} active assemblies in motion`
     : "No active assemblies running"}
 </div>
```

```
 <div className="h-4 w-px bg-[hsl(var(--border))]" />

 <div className="text-xs text-[hsl(var(--muted-foreground))]">
   {stalledCount > 0
     ? `${stalledCount} stalled assemblies need attention`
     : "No stalled assemblies detected"}
 </div>
</GlassPanel>
```

Why this works

It gives the page a top-level “system posture” voice using only data that already exists in the page model.

---

7. Visual Rules

Card style

Use existing `GlassPanel solid` treatment so the overview layer matches the page’s current visual system. The page already relies on `GlassPanel` for the empty state and uses the same system styling language throughout.

Card density

- compact
- one primary number
- one short label
- one small supporting line

Do not overload with charts yet.

Click affordance

Cards should feel like shortcuts, not just analytics tiles.

---

8. Data Integrity Rules

Use only current real data for Pass 4

Safe to derive now:

- total count
- running count
- completed count
- failed count
- queued count
- stalled count
- slow count

Do not fake yet

Do not add cards for:

- Family count
- In Use
- Unowned
- Deprecated
- Lifecycle mix

Those belong in a later pass once the table fields actually exist.

---

9. Placement Relative to Filters

Use this exact structure in the render tree:

```
<div className="space-y-6 animate-fade-in">
 {/* header */}

 {/* overview cards */}

 {/* optional health banner */}

 {/* status chips */}
```

```
{/* saved views */}

{/* secondary filters */}

{/* loading / empty / table */}
</div>
```

This order is important because it makes the page read top-down as:

1. identity
2. health
3. control
4. records

---

10. Pass 4 Acceptance Criteria

Pass 4 is complete when:

- the page has an overview card strip under the header
- cards use only currently available real or directly derived data
- cards are clickable and affect filters/views
- stalled count is derived from current stall detection, not a fake placeholder
- the page feels useful before the table is read
- no backend change is required for this pass.

---

11. Result of Pass 4

After this pass, the page stops opening like a plain filtered list and starts opening like a control surface. That is the right evolution point before you add richer entity-aware layers such as family summaries, ownership rollups, lifecycle mix, and usage metrics.

Pass 5: exact quick-detail

Pass 5 — Exact Quick-Detail Drawer and Row Expansion Block

Pass Goal

Add a fast inspection layer so users can preview an assembly's key details from the Assemblies page without leaving the table.

This pass should let the user quickly inspect:

- what the assembly is
- current execution status
- lifecycle state
- usage state
- family assignment
- owner status
- pipeline stage snapshot
- recent timing context
- next actions

while preserving the current full navigation path into `/assembly/:id`.

runs

---

1. Interaction Model

Current

The row click goes directly to:

- `/assembly/:id`
  runs

Pass 5 Model

Split interactions into two layers:

Primary row click

Open a **quick-detail drawer**

Explicit open action

Keep the current external/open button to navigate to:

- `/assembly/:id`

Why

This creates:

- fast scan workflow inside the page
- full detail workflow when needed

That is a better command-surface pattern than forcing full navigation for every inspection.

---

2. Drawer vs Expandable Row

Recommendation

Use a **right-side drawer**, not inline row expansion.

Why drawer is better

For this page, the drawer is the stronger option because:

- the table is already dense
- expanded rows will break scan rhythm
- drawer keeps context while preserving table readability
- drawer gives enough room for family, owner, usage, and relationship fields later

So Pass 5 should be a **Quick Assembly Drawer**.

---

3. New State Block

Add this state near the existing page state:

```
const [selectedAssembly, setSelectedAssembly] = useState<Assembly | null>(null);
const [drawerOpen, setDrawerOpen] = useState(false);
```

Optional derived helper

```
const openAssemblyDrawer = (assembly: Assembly) => {
 setSelectedAssembly(assembly);
 setDrawerOpen(true);
};

const closeAssemblyDrawer = () => {
 setDrawerOpen(false);
 setSelectedAssembly(null);
};
```

---

4. Row Click Rewrite

Current behavior

Rows navigate directly to `/assembly/:id`.

runs

Pass 5 change

Replace row click with drawer open behavior.

Exact row click update

```
onClick={() => openAssemblyDrawer(a)}
```

Important

Do **not** remove the explicit open button in the Actions column. That button becomes the full-page navigation path.

---

5. Actions Column Update

Keep

- open assembly
- delete

Rule

The open action now has a clearer role:

- row click = inspect
- open button = navigate

That is a cleaner interaction split.

Exact action button

```
<button
 onClick={(e) => {
   e.stopPropagation();
   setLocation(`/assembly/${a.id}`);
 }}
 className="p-1.5 rounded hover:bg-[hsl(var(--accent))] transition-colors"
 title="Open Assembly"
>
 <ExternalLink className="w-3.5 h-3.5 text-[hsl(var(--muted-foreground))]" />
</button>
```

---

6. Drawer Container Block

Add this near the bottom of the page render, after the table/empty state:

```
{drawerOpen && selectedAssembly && (
 <div className="fixed inset-0 z-50 flex justify-end">
   <div
     className="absolute inset-0 bg-black/40 backdrop-blur-[1px]"
```

```
    onClick={closeAssemblyDrawer}
  />

  <div className="relative w-full max-w-xl h-full bg-[hsl(var(--background))] border-l
border-[hsl(var(--border))] shadow-2xl">
    {/* drawer content */}
  </div>
</div>
)}
```

Why this structure

- fast overlay
- easy dismiss
- keeps page context visible underneath
- no routing change required

---

7. Drawer Header Block

The drawer header should immediately identify the assembly and provide the two main actions.

Exact header block

```
<div className="flex items-start justify-between px-5 py-4 border-b border-[hsl(var(--border))]">
 <div className="min-w-0">
   <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
     Assembly
   </p>
   <h2 className="mt-1 text-lg font-semibold text-[hsl(var(--foreground))] truncate">
     {selectedAssembly.projectName}
   </h2>
   {selectedAssembly.idea && (
     <p className="mt-1 text-sm text-[hsl(var(--muted-foreground))] line-clamp-2">
       {selectedAssembly.idea}
     </p>
   )}
 </div>

 <div className="flex items-center gap-2 ml-4">
   <button
```

```
    onClick={() => setLocation(`/assembly/${selectedAssembly.id}`)}
    className="px-3 py-1.5 rounded-md text-xs font-medium border border-[hsl(var(--border))]
hover:bg-[hsl(var(--accent))]"
  >
    Open Assembly
  </button>
  <button
    onClick={closeAssemblyDrawer}
    className="p-2 rounded-md hover:bg-[hsl(var(--accent))]"
    title="Close"
  >
    <X className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
  </button>
 </div>
</div>
```

---

8. Drawer Content Sections

Use a stacked layout with **5 sections**:

1. Status and activity
2. Registry metadata
3. Pipeline snapshot
4. Timing and recency
5. Actions

This order mirrors how a user thinks:

- is it okay?
- what is it?
- where is it in pipeline terms?
- when did it move?
- what can I do?

---

9. Section 1 — Status and Activity

This section should show the most important current posture first.

Exact block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
  Status
 </p>

 <div className="grid grid-cols-2 gap-3">
  <GlassPanel solid className="p-3">
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Execution
   </p>
   <p className="mt-2 text-sm font-medium text-[hsl(var(--foreground))] capitalize">
    {selectedAssembly.status}
   </p>
  </GlassPanel>

  <GlassPanel solid className="p-3">
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Lifecycle
   </p>
   <p className="mt-2 text-sm font-medium text-[hsl(var(--foreground))] capitalize">
    {(selectedAssembly as any).lifecycleState ||
     (selectedAssembly.status === "running"
      ? "active"
      : selectedAssembly.status === "queued"
      ? "draft"
      : selectedAssembly.status === "completed"
      ? "available"
      : "attention")}
   </p>
  </GlassPanel>

  <GlassPanel solid className="p-3">
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Usage
   </p>
   <p className="mt-2 text-sm font-medium text-[hsl(var(--foreground))] capitalize">
    {(selectedAssembly as any).usageState ||
     (selectedAssembly.status === "running"
      ? "live"
      : selectedAssembly.status === "completed"
      ? "idle"
      : selectedAssembly.status === "queued"
```

```
          ? "pending"
          : "blocked")}
    </p>
  </GlassPanel>

  <GlassPanel solid className="p-3">
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Risk
    </p>
    <p className="mt-2 text-sm font-medium text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).riskLevel || "Normal"}
    </p>
  </GlassPanel>
 </div>
</div>
```

---

10. Section 2 — Registry Metadata

This is where the new Assemblies-page meaning starts to show.

Exact block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
  Registry
 </p>

 <div className="grid grid-cols-2 gap-x-4 gap-y-3 text-sm">
  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Family
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).familyName || "Unassigned"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Owner
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
```

```
      {(selectedAssembly as any).ownerName || "Unassigned"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Control Plane
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).controlPlane || "Not set"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Preset
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {selectedAssembly.preset || "default"}
    </p>
  </div>
 </div>
</div>
```

---

11. Section 3 — Pipeline Snapshot

The current page already uses `StageRail`, and that should be reused here.

runs

Exact block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
   Pipeline Snapshot
 </p>

 <StageRail stages={parseStagesFromAssembly((selectedAssembly as any).latestStages)} />
</div>
```

Optional follow-up

Later, this section can show:

- current stage name
- last completed stage
- stall duration
- verification state

But not required for Pass 5.

---

12. Section 4 — Timing and Recency

Use the current timing helpers already present in the page: `formatDuration`, `formatDate`, and the existing elapsed logic pattern.

Exact block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
  Timing
 </p>

 <div className="grid grid-cols-2 gap-x-4 gap-y-3 text-sm">
  <div>
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Updated
   </p>
   <p className="mt-1 text-[hsl(var(--foreground))]">
    {formatDate(selectedAssembly.updatedAt)}
   </p>
  </div>

  <div>
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Created
   </p>
   <p className="mt-1 text-[hsl(var(--foreground))]">
    {formatDate(selectedAssembly.createdAt)}
   </p>
  </div>

  <div>
```

```
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Started
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {selectedAssembly.startedAt
        ? formatDate(selectedAssembly.startedAt)
        : "Not started"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Completed
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {selectedAssembly.completedAt
        ? formatDate(selectedAssembly.completedAt)
        : "Not completed"}
    </p>
  </div>
 </div>
</div>
```

---

13. Section 5 — Drawer Actions

This section gives the user a few immediate next steps without requiring more page chrome elsewhere.

Exact block

```
<div className="px-5 py-4">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
   Actions
 </p>

 <div className="flex flex-wrap gap-2">
   <button
     onClick={() => setLocation(`/assembly/${selectedAssembly.id}`)}
     className="px-3 py-2 rounded-md bg-[hsl(var(--primary))]
text-[hsl(var(--primary-foreground))] text-sm font-medium hover:opacity-90"
   >
     Open Assembly
```

```
  </button>

  <button
    onClick={closeAssemblyDrawer}
    className="px-3 py-2 rounded-md border border-[hsl(var(--border))] text-sm font-medium
hover:bg-[hsl(var(--accent))]"
  >
    Close
  </button>
 </div>
</div>
```

---

14. Recommended Drawer Shell Layout

Wrap the drawer body in a scroll container:

```
<div className="h-full overflow-y-auto">
 {/* header */}
 {/* status */}
 {/* registry */}
 {/* pipeline */}
 {/* timing */}
 {/* actions */}
</div>
```

This is important because the drawer will grow in later passes when you add:

- family summaries
- dependency lists
- related assemblies
- usage metrics
- approval state
- assigned agents

---

15. Optional Row Preview Trigger

If you want to preserve full row navigation instead of changing the row click behavior yet, use a dedicated preview icon in the Actions column.

Alternative action split

- row click = navigate
- eye/info button = drawer preview
- external/open button = navigate
- trash = delete

That is a valid alternative, but my stronger recommendation is still:

- row click = preview drawer
- action button = open full assembly

because it supports scanning better.

---

16. Pass 5 Acceptance Criteria

Pass 5 is complete when:

- user can open a quick-detail drawer from the table
- drawer shows assembly identity, status, registry metadata, pipeline snapshot, and timing
- full-page navigation to `/assembly/:id` still exists
- drawer can be closed by overlay, button, or close icon
- no backend change is required for the base drawer
- the drawer reuses current page data and helpers where possible.

---

17. Result of Pass 5

After this pass, the page becomes much more usable operationally. Users will be able to scan, inspect, and decide without leaving the Assemblies page every time. That is a major step toward making this feel like a command surface instead of just a list with links.

Pass 6: exact family grouping

Pass 6 — Exact Family Grouping and Family Summary Block

Pass Goal

Turn the Assemblies page from a flat list into a grouped operational view where related assemblies can be understood and managed together.

This pass should let users:

- see which assemblies belong together
- collapse and expand family groups
- understand family health at a glance
- identify unassigned assemblies quickly
- treat assemblies as parts of systems, not isolated records

---

1. Grouping Model

Current

The page renders a single flat `filtered.map(...)` table body.

runs

Pass 6 model

Render the table as grouped sections:

- one **family header row**
- followed by its **assembly rows**
- plus one explicit **Unassigned** group

Why

A Family column alone helps identification, but grouping is what changes the management experience.

---

2. Family Data Assumption

Pass 6 requirement

Use these fields when available:

- `familyId`
- `familyName`
- `familyType`
- `familyHealth`

Safe fallback

Until full family metadata exists, group by:

- `(a as any).familyName || "Unassigned"`

That makes the grouping feature shippable before the backend is fully enriched.

---

3. New State Block

Add group-mode and collapse state near the other filter/page state:

```
const [groupByFamily, setGroupByFamily] = useState(true);
const [collapsedFamilies, setCollapsedFamilies] = useState<Record<string, boolean>>({});
```

Toggle helper

```
const toggleFamilyCollapse = (familyKey: string) => {
 setCollapsedFamilies((prev) => ({
   ...prev,
   [familyKey]: !prev[familyKey],
 }));
```

```
};
```

Why this is enough

You do not need a more complex tree state yet. Family grouping is one level deep.

---

4. Family Group Derivation

Create a grouped data structure from the already-filtered list.

Exact grouping block

```
const groupedFamilies = filtered.reduce<Record<string, Assembly[]>>((acc, assembly) => {
 const familyKey = (assembly as any).familyName || "Unassigned";
 if (!acc[familyKey]) acc[familyKey] = [];
 acc[familyKey].push(assembly);
 return acc;
}, {});
```

Sorted group entries

```
const familyEntries = Object.entries(groupedFamilies).sort(([a], [b]) => {
 if (a === "Unassigned") return 1;
 if (b === "Unassigned") return -1;
 return a.localeCompare(b);
});
```

Why sort this way

Assigned families should feel primary.
Unassigned should always remain visible, but secondary.

---

5. Family Summary Derivation

Each family header needs a summary line.

Exact summary helper

```
const getFamilySummary = (assemblies: Assembly[]) => {
 const running = assemblies.filter((a) => a.status === "running").length;
 const completed = assemblies.filter((a) => a.status === "completed").length;
 const failed = assemblies.filter((a) => a.status === "failed").length;
 const queued = assemblies.filter((a) => a.status === "queued").length;

 return {
   total: assemblies.length,
   running,
   completed,
   failed,
   queued,
 };
};
```

Why

This uses only data the page already has today: assembly status. That makes the grouping layer immediately usable without waiting for family metrics APIs. The page already derives counts from `assemblies` in the same way.

runs

---

6. Family Health Heuristic

Until real `familyHealth` is supplied, derive a temporary family health label from member statuses.

Exact helper

```
const getFamilyHealth = (assemblies: Assembly[]) => {
 const hasFailed = assemblies.some((a) => a.status === "failed");
 const hasRunning = assemblies.some((a) => a.status === "running");
 const hasQueued = assemblies.some((a) => a.status === "queued");

 if (hasFailed) return "At Risk";
 if (hasRunning) return "Active";
 if (hasQueued) return "Pending";
```

```
 return "Stable";
};
```

Why

This is a good temporary operational signal:

- failed member means the family is at risk
- running means active
- queued means pending
- otherwise stable

Later this can be replaced by actual family health.

---

7. Grouping Toggle in UI

Add a compact grouping toggle near the saved views / filters row.

Exact block

```
<div className="flex items-center gap-2">
 <button
   onClick={() => setGroupByFamily(false)}
   className={`px-3 py-1.5 rounded-md text-xs border ${
     !groupByFamily
       ? "bg-[hsl(var(--primary)/0.15)] text-[hsl(var(--primary))] border-[hsl(var(--primary)/0.25)]"
       : "border-[hsl(var(--border))] text-[hsl(var(--muted-foreground))]"
   }`}
 >
   Flat View
 </button>

 <button
   onClick={() => setGroupByFamily(true)}
   className={`px-3 py-1.5 rounded-md text-xs border ${
     groupByFamily
       ? "bg-[hsl(var(--primary)/0.15)] text-[hsl(var(--primary))] border-[hsl(var(--primary)/0.25)]"
       : "border-[hsl(var(--border))] text-[hsl(var(--muted-foreground))]"
   }`}
 >
```

```
  Family View
 </button>
</div>
```

Why

You should not force grouped view permanently yet.
Give the user both:

- flat operational scan
- grouped ecosystem scan

---

8. Table Rendering Strategy

Current

One `<tbody>` with a direct `filtered.map(...)`.

runs

Pass 6

Branch rendering:

- if `groupByFamily === false`: keep the current flat row render
- if `groupByFamily === true`: render family headers + child rows

That avoids destabilizing the page while you test grouped mode.

---

9. Family Header Row

Each family needs a visual summary row spanning the table.

Exact family header row

```
<tr className="bg-[hsl(var(--muted)/0.45)] border-t border-[hsl(var(--border))]">
```

```
<td colSpan={9} className="px-4 py-3">
  <div className="flex items-center justify-between gap-4">
    <div className="flex items-center gap-3 min-w-0">
      <button
        onClick={() => toggleFamilyCollapse(familyName)}
        className="p-1 rounded hover:bg-[hsl(var(--accent))]"
      >
        {collapsedFamilies[familyName] ? (
          <ChevronRight className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
        ) : (
          <ChevronDown className="w-4 h-4 text-[hsl(var(--muted-foreground))]" />
        )}
      </button>

      <div className="min-w-0">
        <div className="flex items-center gap-2 flex-wrap">
          <span className="text-sm font-semibold text-[hsl(var(--foreground))]">
            {familyName}
          </span>

          <span className="px-2 py-0.5 rounded-full text-[10px] border border-[hsl(var(--border))]
text-[hsl(var(--muted-foreground))]">
            {familyName === "Unassigned"
              ? "Unassigned"
              : ((familyAssemblies[0] as any).familyType || "Family")}
          </span>

          <span className="px-2 py-0.5 rounded-full text-[10px] bg-[hsl(var(--background))]
border border-[hsl(var(--border))] text-[hsl(var(--foreground))]">
            {familyHealth}
          </span>
        </div>

        <div className="mt-1 flex items-center gap-3 flex-wrap text-[11px]
text-[hsl(var(--muted-foreground))]">
          <span>{summary.total} assemblies</span>
          <span>{summary.running} running</span>
          <span>{summary.completed} completed</span>
          <span>{summary.failed} failed</span>
          <span>{summary.queued} queued</span>
        </div>
      </div>
    </div>
  </div>
```

</td>
</tr>

Required imports

Add:

- `ChevronDown`
- `ChevronRight`

from `lucide-react`.

---

10. Child Row Rendering Under Each Family

Under each family header, render the same row model you already use for assemblies.

Rule

Do not create a second row design for grouped mode.
Reuse the existing assembly row.

Why

This keeps behavior consistent:

- same click handling
- same status rendering
- same pipeline rail
- same actions

The current row design already has the right operational base.

runs

---

11. Exact Grouped Table Body Pattern

Here is the render structure for grouped mode:

```
<tbody>
 {familyEntries.map(([familyName, familyAssemblies]) => {
   const summary = getFamilySummary(familyAssemblies);
   const familyHealth =
     (familyAssemblies[0] as any).familyHealth || getFamilyHealth(familyAssemblies);
   const isCollapsed = !!collapsedFamilies[familyName];

   return (
     <Fragment key={familyName}>
       <tr className="bg-[hsl(var(--muted)/0.45)] border-t border-[hsl(var(--border))]">
         <td colSpan={9} className="px-4 py-3">
           {/* family header content */}
         </td>
       </tr>

       {!isCollapsed &&
         familyAssemblies.map((a) => {
           const statusEntry =
             a.status === "running"
               ? pipelineStatus?.activeRuns?.find((s) => s.assemblyId === a.id)
               : undefined;

           const stallLevel = statusEntry
             ? getStallLevel(statusEntry.stalledMs)
             : "none";

           return (
             <tr
               key={a.id}
               className="border-t border-[hsl(var(--border)/0.5)] hover:bg-[hsl(var(--accent)/0.5)]
cursor-pointer transition-colors"
               onClick={() => openAssemblyDrawer(a)}
             >
               {/* existing row cells */}
             </tr>
           );
         })}
     </Fragment>
   );
```

```
 })}
</tbody>
```

Required import

Add:

```
import { Fragment } from "react";
```

---

12. Unassigned Group Rules

The **Unassigned** group should be treated as a real group, not hidden.

Why

This is one of the highest-value management signals:

- assemblies missing organization
- portfolio drift
- likely cleanup or classification work needed

UI behavior

- always show `Unassigned`
- place it last
- do not default it to collapsed
- use muted or warning-adjacent visual emphasis later if desired

---

13. Optional Family Summary Cards Above Table

Once family grouping exists, you can optionally add a thin family summary row above the table.

First-pass optional metrics

- Total Families
- Active Families
- At Risk Families
- Unassigned Assemblies

Exact derivation

```
const totalFamilies = familyEntries.filter(([name]) => name !== "Unassigned").length;
const activeFamilies = familyEntries.filter(([, group]) =>
 group.some((a) => a.status === "running")
).length;
const atRiskFamilies = familyEntries.filter(([, group]) =>
 group.some((a) => a.status === "failed")
).length;
const unassignedAssemblies = groupedFamilies["Unassigned"]?.length ?? 0;
```

This is optional for Pass 6, but it pairs very well with family mode.

---

14. Filter Interaction Rules with Family View

Family grouping should respect all existing filters.

Meaning

The flow stays:

1. assemblies fetched
2. filters applied
3. resulting rows grouped into families

Do not

Do not group first and then filter.
That would create misleading group counts and empty groups.

---

15. Family Collapse Defaults

Recommendation

Default all families to expanded.

Why

You want the grouped view to feel informative immediately.
Collapsed-by-default makes the page feel hidden and inactive.

Later you can preserve collapse state across sessions if needed.

---

16. Family Header Actions — Future Hooks

Do not overload Pass 6, but design the family header so these can be added later:

- open family view
- assign selected assembly to family
- bulk lifecycle update
- bulk owner assignment
- family health detail
- family topology preview

So leave space on the right side of the header row for future controls.

---

17. Pass 6 Acceptance Criteria

Pass 6 is complete when:

- the page supports **Flat View** and **Family View**
- filtered assemblies can be grouped by family name
- each family renders a group header with summary counts
- each family can be collapsed and expanded

- unassigned assemblies are shown in an explicit **Unassigned** group
- grouped mode reuses the current assembly row behavior and actions
- family health can be shown using a temporary derived heuristic when real family health is absent

The current page already provides the flat row rendering base, status model, stage rail, and actions that this grouped mode should reuse.

---

18. Result of Pass 6

After this pass, the Assemblies page stops feeling like a list of independent jobs and starts feeling like a system registry. That is the real value of families: they add operational context, ownership context, and ecosystem context without requiring a fully separate topology page yet.

Pass 7: exact ownership

Pass 7 — Exact Ownership, Control-Plane, and Agent Assignment Block

Pass Goal

Add a responsibility layer so each assembly can show:

- who owns it
- what team or control plane governs it
- what agents are assigned to it
- which assemblies are unowned
- which assemblies are missing operational governance

This pass should make the Assemblies page answer:

- who is responsible for this assembly?
- what governing plane is it part of?
- which automation or agent is attached to it?
- what needs assignment cleanup?

---

1. Responsibility Model

Current

The page shows:

- assembly identity
- execution status
- pipeline stage rail
- timing context
- updated timestamp
- row actions.

Missing

The page does **not** show:

- owner
- team
- control plane
- assigned agents
- assignment health

Pass 7 target

Every assembly row should expose a responsibility layer with at least:

- owner
- control plane
- assigned agents summary

That responsibility data should also be visible in:

- the quick-detail drawer

- filters
- saved views
- grouped/family scan

---

2. Minimum Data Additions

Add these fields to the page model when available:

```
ownerName?: string | null;
ownerId?: string | null;
teamName?: string | null;
controlPlane?: string | null;
assignedAgents?: Array<{
 id: string;
 name: string;
 role?: string;
 status?: "active" | "idle" | "paused";
}> | null;
```

Safe fallback rule

Until the backend is fully updated, the page should safely render:

- `ownerName || "Unassigned"`
- `controlPlane || "No control plane"`
- `assignedAgents?.length || 0`

This matches the same progressive approach used in the earlier passes for family/lifecycle/usage placeholders.

---

3. Ownership Column Upgrade

Pass 2 added an **Owner** column conceptually. In Pass 7, that column becomes real.

Owner cell contents

The Owner column should display:

- owner name
- team name
- control-plane tag
- agent-count summary

Exact Owner cell block

```
<td className="px-4 py-3 hidden xl:table-cell">
 <div className="flex flex-col gap-1 min-w-0">
   <div className="flex items-center gap-2 min-w-0">
     <span className="text-xs text-[hsl(var(--foreground))] truncate">
       {(a as any).ownerName || "Unassigned"}
     </span>

     {!(a as any).ownerName && (
       <span className="px-1.5 py-0.5 rounded-full text-[10px] border
border-[hsl(var(--status-warning)/0.35)] text-[hsl(var(--status-warning))]">
         Needs owner
       </span>
     )}
   </div>

   <div className="flex items-center gap-2 flex-wrap">
     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       {(a as any).teamName || "No team"}
     </span>

     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">•</span>

     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       {(a as any).controlPlane || "No control plane"}
```

```
    </span>
   </div>

   <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
    {Array.isArray((a as any).assignedAgents)
      ? `${(a as any).assignedAgents.length} agent${(a as any).assignedAgents.length === 1 ? ""
: "s"}`
      : "No agents"}
   </span>
 </div>
</td>
```

Why this shape

A single-line owner field is not enough. The point of this pass is not just “who owns it,” but “what responsibility system surrounds it.”

---

4. New Responsibility Summary Helpers

Add these above the render block so cards, filters, and badges can reuse them.

Exact helpers

```
const getOwnerName = (assembly: Assembly) => (assembly as any).ownerName || null;
const getTeamName = (assembly: Assembly) => (assembly as any).teamName || null;
const getControlPlane = (assembly: Assembly) => (assembly as any).controlPlane || null;
const getAssignedAgents = (assembly: Assembly) =>
 Array.isArray((assembly as any).assignedAgents) ? (assembly as any).assignedAgents : [];
```

Optional assignment health helper

```
const getAssignmentHealth = (assembly: Assembly) => {
 const owner = getOwnerName(assembly);
 const controlPlane = getControlPlane(assembly);
 const agents = getAssignedAgents(assembly);

 if (!owner && !controlPlane && agents.length === 0) return "unassigned";
 if (!owner || !controlPlane) return "partial";
 return "assigned";
};
```

Why

This gives you a clean way to build:

- filters
- summary cards
- warning chips
- drawer sections

without repeating null-check logic everywhere.

---

5. Ownership Filter Expansion

The page already moved toward a multi-filter model in Pass 3. This pass makes the ownership filter genuinely useful.

Replace simple owner filter type

Current planned type was:

- all
- assigned
- unassigned

Expand it to:

```
type OwnerFilter =
 | "all"
 | "assigned"
 | "unassigned"
 | "partial"
 | "with_agents"
 | "without_agents"
 | "no_control_plane";
```

Exact filter select block

```
<select
 value={ownerFilter}
 onChange={(e) => setOwnerFilter(e.target.value as OwnerFilter)}
 className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
>
 <option value="all">All Ownership</option>
 <option value="assigned">Fully Assigned</option>
 <option value="partial">Partially Assigned</option>
 <option value="unassigned">Unowned</option>
 <option value="with_agents">With Agents</option>
 <option value="without_agents">No Agents</option>
 <option value="no_control_plane">No Control Plane</option>
</select>
```

Why

This is more operationally useful than a simple assigned/unassigned split.

---

6. Derived Filter Logic Update

Extend the layered filter pipeline with ownership-aware logic.

Exact ownership matching block

```
const assignmentHealth = getAssignmentHealth(a);
const ownerName = getOwnerName(a);
const controlPlane = getControlPlane(a);
const assignedAgents = getAssignedAgents(a);

const matchesOwner =
 ownerFilter === "all"
   ? true
   : ownerFilter === "assigned"
   ? assignmentHealth === "assigned"
   : ownerFilter === "partial"
   ? assignmentHealth === "partial"
   : ownerFilter === "unassigned"
```

```
  ? assignmentHealth === "unassigned"
  : ownerFilter === "with_agents"
  ? assignedAgents.length > 0
  : ownerFilter === "without_agents"
  ? assignedAgents.length === 0
  : !controlPlane;
```

Important

This should replace the earlier simpler owner-filter logic from Pass 3.

---

7. Saved Views Expansion

The saved-views layer should now support accountability-focused shortcuts.

Add these views

Extend the saved views with:

- `unowned`
- `no_control_plane`
- `agentless`
- `partial_assignment`

Exact expanded saved view type

```
type SavedView =
 | "all"
 | "active"
 | "failed"
 | "stalled"
 | "in_use"
 | "unowned"
 | "no_family"
 | "deprecated_candidates"
```

```
| "at_risk"
| "no_control_plane"
| "agentless"
| "partial_assignment";
```

Exact matching block additions

```
: savedView === "unowned"
? !ownerName
: savedView === "no_control_plane"
? !controlPlane
: savedView === "agentless"
? assignedAgents.length === 0
: savedView === "partial_assignment"
? assignmentHealth === "partial"
```

Why

These are the views that turn the page into a responsibility cleanup tool.

---

8. Responsibility Summary Cards

Pass 4 added a status-focused overview strip. Pass 7 should add accountability cards once owner/control-plane/agent fields exist.

Add these four cards after the existing operational set

- Owned
- Unowned
- With Agents
- Missing Control Plane

Exact derived metrics

```
const ownedCount = assemblies.filter((a) => !!getOwnerName(a)).length;
```

```
const unownedCount = assemblies.filter((a) => !getOwnerName(a)).length;
const withAgentsCount = assemblies.filter((a) => getAssignedAgents(a).length > 0).length;
const missingControlPlaneCount = assemblies.filter((a) => !getControlPlane(a)).length;
```

Card click mapping

- Owned → `setOwnerFilter("assigned")`
- Unowned → `setSavedView("unowned")`
- With Agents → `setOwnerFilter("with_agents")`
- Missing Control Plane → `setSavedView("no_control_plane")`

Why

These cards help users spot governance gaps quickly without scanning rows.

---

9. Responsibility Badge in Assembly Cell

A small accountability signal should appear directly in the Assembly identity cell for fast scan quality.

Exact badge block

Add this under the preset/secondary metadata in the Assembly cell:

```
<div className="flex items-center gap-2 flex-wrap">
 {getAssignmentHealth(a) === "unassigned" && (
   <span className="px-1.5 py-0.5 rounded-full text-[10px] border
border-[hsl(var(--status-warning)/0.35)] text-[hsl(var(--status-warning))]">
     Unassigned
   </span>
 )}

 {getAssignmentHealth(a) === "partial" && (
```

```
  <span className="px-1.5 py-0.5 rounded-full text-[10px] border
border-[hsl(var(--status-warning)/0.35)] text-[hsl(var(--status-warning))]">
    Partial
  </span>
 )}

 {getAssignmentHealth(a) === "assigned" && (
  <span className="px-1.5 py-0.5 rounded-full text-[10px] border border-[hsl(var(--border))]
text-[hsl(var(--muted-foreground))]">
    Assigned
  </span>
 )}
</div>
```

Why

This gives immediate scan value even when the Owner column is hidden on smaller breakpoints.

---

10. Quick-Detail Drawer Upgrade

The quick-detail drawer from Pass 5 should be extended with a dedicated **Responsibility** section.

Exact drawer section

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
  Responsibility
 </p>

 <div className="grid grid-cols-2 gap-x-4 gap-y-3 text-sm">
  <div>
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Owner
   </p>
   <p className="mt-1 text-[hsl(var(--foreground))]">
    {(selectedAssembly as any).ownerName || "Unassigned"}
   </p>
  </div>
```

```
  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Team
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).teamName || "Not set"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Control Plane
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).controlPlane || "No control plane"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Assignment Health
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
      {getAssignmentHealth(selectedAssembly)}
    </p>
  </div>
 </div>

 <div className="mt-4">
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]
mb-2">
     Assigned Agents
   </p>

   <div className="flex flex-wrap gap-2">
     {getAssignedAgents(selectedAssembly).length > 0 ? (
       getAssignedAgents(selectedAssembly).map((agent: any) => (
         <span
           key={agent.id}
           className="px-2 py-1 rounded-full text-[11px] border border-[hsl(var(--border))]
text-[hsl(var(--foreground))]"
         >
           {agent.name}
```

```
          {agent.role ? ` · ${agent.role}` : ""}
        </span>
      ))
    ) : (
      <span className="text-xs text-[hsl(var(--muted-foreground))]">
        No agents assigned
      </span>
    )}
  </div>
 </div>
</div>
```

Why

Ownership needs its own dedicated section, not just one row in Registry metadata.

---

11. Family Header Responsibility Rollup

In Family View, family headers should start surfacing accountability quality too.

Exact derived metrics

```
const getFamilyResponsibilitySummary = (assemblies: Assembly[]) => {
 const owned = assemblies.filter((a) => !!getOwnerName(a)).length;
 const withControlPlane = assemblies.filter((a) => !!getControlPlane(a)).length;
 const withAgents = assemblies.filter((a) => getAssignedAgents(a).length > 0).length;

 return {
   owned,
   withControlPlane,
   withAgents,
 };
};
```

Add to family header summary line

```
<span>{responsibility.owned}/{summary.total} owned</span>
<span>{responsibility.withControlPlane}/{summary.total} governed</span>
<span>{responsibility.withAgents}/{summary.total} with agents</span>
```

Why

Once families exist, governance quality should be visible at the family level too.

---

12. Row-Level Visual Warning Rules

Responsibility issues should affect row scan quality subtly.

Suggested rules

- no owner → small warning pill
- no control plane → muted warning text in Owner cell
- no agents → neutral “No agents”
- fully assigned → no warning

Do not

Do not turn every missing assignment into aggressive red failure styling.
These are governance gaps, not necessarily runtime failures.

That distinction matters.

---

13. Control Plane as a First-Class Concept

Do not treat control plane as just metadata text.

On this page, control plane should function as:

- a visible governance label
- a filterable operational domain

- a future grouping/saved-view axis

Future-ready values might include

- orchestration
- verification
- maintenance
- deployment
- analytics
- runtime oversight

But for this pass, render it as a free string field.

---

14. Agent Assignment Display Rules

Agent assignment should remain compact in the table.

Table rule

Do not list all agent names inline in the row by default.

Use:

- `0 agents`
- `1 agent`
- `3 agents`

Drawer rule

List actual names and roles there.

Why

That keeps the table readable while still exposing detail where it belongs.

---

15. Pass 7 Acceptance Criteria

Pass 7 is complete when:

- the Owner column shows real responsibility structure
- unowned assemblies are visually identifiable
- control plane visibility exists in the row and drawer
- assigned-agent summaries exist in the row and drawer
- owner filters can distinguish full, partial, and missing assignment
- saved views can surface responsibility cleanup states
- family headers can summarize responsibility coverage
- the page becomes useful for governance and accountability, not just monitoring

This pass builds on the current page's existing assembly identity, status model, actions, and row structure, which already give it a strong operational base.

---

16. Result of Pass 7

After this pass, the Assemblies page starts functioning like a true operational registry. Users will not just see what exists and what is running — they will see who owns what, what lacks governance, and where assignment cleanup is needed.

Pass 8: Exact Lifecycle,

Pass 8 — Exact Lifecycle, Deprecation, and Retirement-Management Block

Pass Goal

Add a long-term lifecycle layer so assemblies can be managed after pipeline execution is over.

This pass should let users understand:

- whether an assembly is merely completed or actually active
- whether it is still in use or drifting out
- whether it is deprecated, archived, or a retirement candidate
- which assemblies are safe to phase out
- which assemblies are dangerously still in use despite deprecation

That distinction does not exist in the current page, which is why this pass matters.

---

1. Lifecycle Model

Current

The page only has execution status:

- `running`
- `completed`
- `failed`
- `queued`

Problem

Execution status answers:

- what happened in the pipeline?

It does **not** answer:

- what is this assembly’s actual operational life state?

Pass 8 model

Keep execution status, but add a second axis:

Execution status

- queued
- running
- completed
- failed

Lifecycle state

- draft
- active
- in_use
- degraded
- deprecated
- archived

This is the core rule of Pass 8:
pipeline state and lifecycle state must coexist, not replace each other.

---

2. Minimum Data Additions

Add these fields when available:

```
lifecycleState?: "draft" | "active" | "in_use" | "degraded" | "deprecated" | "archived" | null;
lifecycleUpdatedAt?: string | null;
deprecationState?: "none" | "planned" | "announced" | "in_progress" | "completed" | null;
deprecationTargetDate?: string | null;
retirementCandidate?: boolean | null;
usageLastSeenAt?: string | null;
```

Safe fallback rule

Until backend fields exist, derive temporary lifecycle values from current status:

- `queued` → `draft`
- `running` → `active`
- `completed` → `active`
- `failed` → `degraded`

That gives you a workable first implementation without blocking on backend changes.

---

3. Lifecycle Helper Functions

Add these above render so the logic stays centralized.

Exact lifecycle helper

```
const getLifecycleState = (assembly: Assembly) => {
 const explicit = (assembly as any).lifecycleState;
 if (explicit) return explicit;

 if (assembly.status === "queued") return "draft";
 if (assembly.status === "running") return "active";
 if (assembly.status === "completed") return "active";
 return "degraded";
};
```

Deprecation helper

```
const getDeprecationState = (assembly: Assembly) =>
 (assembly as any).deprecationState || "none";
```

Retirement-candidate helper

```
const isRetirementCandidate = (assembly: Assembly) => {
 const explicit = (assembly as any).retirementCandidate;
 if (typeof explicit === "boolean") return explicit;

 const lifecycle = getLifecycleState(assembly);
 const usageState = (assembly as any).usageState || null;

 return assembly.status === "completed" &&
   lifecycle !== "active" &&
   lifecycle !== "in_use" &&
```

```
  usageState !== "live";
};
```

Why

This keeps lifecycle/deprecation logic consistent across:

- table cells
- filters
- cards
- drawer
- saved views

---

4. Lifecycle Column Upgrade

Pass 2 introduced the `Lifecycle` column conceptually. Pass 8 makes it real.

Exact Lifecycle cell block

```
<td className="px-4 py-3 hidden lg:table-cell">
 <div className="flex flex-col gap-1">
   <span className="text-xs text-[hsl(var(--foreground))] capitalize">
     {getLifecycleState(a).replace("_", " ")}
   </span>

   {getDeprecationState(a) !== "none" && (
     <span className="text-[10px] text-[hsl(var(--status-warning))] capitalize">
       {getDeprecationState(a).replace("_", " ")}
     </span>
   )}

   {isRetirementCandidate(a) && (
     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       Retirement candidate
     </span>
   )}
 </div>
</td>
```

Why this shape

Lifecycle needs to show more than one thing:

- present state
- deprecation overlay
- retirement hint

That is much more useful than one flat badge.

---

5. Lifecycle Filter Upgrade

The page's earlier multi-filter model should now rely on actual lifecycle logic instead of placeholder strings.

Exact lifecycle filter type

```
type LifecycleFilter =
 | "all"
 | "draft"
 | "active"
 | "in_use"
 | "degraded"
 | "deprecated"
 | "archived"
 | "retirement_candidates";
```

Exact lifecycle select block

```
<select
 value={lifecycleFilter}
 onChange={(e) => setLifecycleFilter(e.target.value as LifecycleFilter)}
 className="px-2.5 py-1.5 rounded-md text-xs bg-[hsl(var(--background))] border
border-[hsl(var(--border))]"
>
 <option value="all">All Lifecycle</option>
 <option value="draft">Draft</option>
 <option value="active">Active</option>
 <option value="in_use">In Use</option>
 <option value="degraded">Degraded</option>
 <option value="deprecated">Deprecated</option>
```

```
 <option value="archived">Archived</option>
 <option value="retirement_candidates">Retirement Candidates</option>
</select>
```

Exact lifecycle matching block

```
const lifecycleState = getLifecycleState(a);
const deprecationState = getDeprecationState(a);
const retirementCandidate = isRetirementCandidate(a);

const matchesLifecycle =
 lifecycleFilter === "all"
   ? true
   : lifecycleFilter === "retirement_candidates"
   ? retirementCandidate
   : lifecycleFilter === "deprecated"
   ? lifecycleState === "deprecated" || deprecationState !== "none"
   : lifecycleState === lifecycleFilter;
```

Why

Deprecation needs to work both as:

- a lifecycle state
- and a managed overlay state

That flexibility matters.

---

6. Saved Views Expansion

Add lifecycle-aware saved views that help with cleanup and retirement workflows.

Add these views

- `deprecated`
- `retirement_candidates`
- `archived`
- `degraded`

Exact SavedView additions

```
type SavedView =
 | "all"
 | "active"
 | "failed"
 | "stalled"
 | "in_use"
 | "unowned"
 | "no_family"
 | "deprecated_candidates"
 | "at_risk"
 | "no_control_plane"
 | "agentless"
 | "partial_assignment"
 | "deprecated"
 | "retirement_candidates"
 | "archived"
 | "degraded";
```

Exact saved-view matching additions

```
: savedView === "deprecated"
? getLifecycleState(a) === "deprecated" || getDeprecationState(a) !== "none"
: savedView === "retirement_candidates"
? isRetirementCandidate(a)
: savedView === "archived"
? getLifecycleState(a) === "archived"
: savedView === "degraded"
? getLifecycleState(a) === "degraded"
```

Why

This gives the page real end-of-life management views, not just active-work views.

---

7. Overview Card Expansion

Pass 4 added operational summary cards based on current counts and stall detection. This pass adds lifecycle-aware management cards.

Add these cards

- Active Lifecycle
- In Use
- Deprecated
- Retirement Candidates
- Archived
- Degraded

Exact derived metrics

```
const activeLifecycleCount = assemblies.filter((a) => getLifecycleState(a) === "active").length;
const inUseCount = assemblies.filter((a) => getLifecycleState(a) === "in_use").length;
const deprecatedCount = assemblies.filter(
 (a) => getLifecycleState(a) === "deprecated" || getDeprecationState(a) !== "none"
).length;
const retirementCandidateCount = assemblies.filter((a) => isRetirementCandidate(a)).length;
const archivedCount = assemblies.filter((a) => getLifecycleState(a) === "archived").length;
const degradedCount = assemblies.filter((a) => getLifecycleState(a) === "degraded").length;
```

Card click mapping

- Active Lifecycle → `setLifecycleFilter("active")`
- In Use → `setLifecycleFilter("in_use")`
- Deprecated → `setSavedView("deprecated")`
- Retirement Candidates → `setSavedView("retirement_candidates")`
- Archived → `setSavedView("archived")`
- Degraded → `setSavedView("degraded")`

Why

These cards turn lifecycle into a top-of-page management concern instead of burying it in the table.

---

8. Assembly Cell Lifecycle Badge

A small lifecycle badge should appear in the Assembly cell so even reduced-width layouts still surface long-term state.

Exact badge block

```
<div className="flex items-center gap-2 flex-wrap">
 <span className="px-1.5 py-0.5 rounded-full text-[10px] border border-[hsl(var(--border))]
text-[hsl(var(--muted-foreground))] capitalize">
   {getLifecycleState(a).replace("_", " ")}
 </span>

 {getDeprecationState(a) !== "none" && (
   <span className="px-1.5 py-0.5 rounded-full text-[10px] border
border-[hsl(var(--status-warning)/0.35)] text-[hsl(var(--status-warning))] capitalize">
     {getDeprecationState(a).replace("_", " ")}
   </span>
 )}

 {isRetirementCandidate(a) && (
   <span className="px-1.5 py-0.5 rounded-full text-[10px] border border-[hsl(var(--border))]
text-[hsl(var(--muted-foreground))]">
     Retirement candidate
   </span>
 )}
</div>
```

Why

This keeps lifecycle visible even when the dedicated Lifecycle column is hidden on smaller breakpoints.

---

9. Quick-Detail Drawer Upgrade

The drawer from Pass 5 should gain a dedicated **Lifecycle** section.

Exact drawer block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
   Lifecycle
 </p>

 <div className="grid grid-cols-2 gap-x-4 gap-y-3 text-sm">
```

```
  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Lifecycle State
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
      {getLifecycleState(selectedAssembly).replace("_", " ")}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Deprecation
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
      {getDeprecationState(selectedAssembly).replace("_", " ")}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Retirement Candidate
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {isRetirementCandidate(selectedAssembly) ? "Yes" : "No"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Lifecycle Updated
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {(selectedAssembly as any).lifecycleUpdatedAt
        ? formatDate((selectedAssembly as any).lifecycleUpdatedAt)
        : "Not tracked"}
    </p>
  </div>
</div>

{(selectedAssembly as any).deprecationTargetDate && (
  <div className="mt-4">
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Deprecation Target
    </p>
```

```
    <p className="mt-1 text-sm text-[hsl(var(--foreground))]">
      {formatDate((selectedAssembly as any).deprecationTargetDate)}
    </p>
  </div>
 )}
</div>
```

Why

Lifecycle is too important to leave as one table field. It needs its own inspection section.

---

10. Family Header Lifecycle Rollup

In Family View, family headers should start surfacing lifecycle mix, not just execution mix.

Exact helper

```
const getFamilyLifecycleSummary = (assemblies: Assembly[]) => {
 const active = assemblies.filter((a) => getLifecycleState(a) === "active").length;
 const inUse = assemblies.filter((a) => getLifecycleState(a) === "in_use").length;
 const deprecated = assemblies.filter(
   (a) => getLifecycleState(a) === "deprecated" || getDeprecationState(a) !== "none"
 ).length;
 const archived = assemblies.filter((a) => getLifecycleState(a) === "archived").length;
 const retirementCandidates = assemblies.filter((a) => isRetirementCandidate(a)).length;

 return {
   active,
   inUse,
   deprecated,
   archived,
   retirementCandidates,
 };
};
```

Add to family summary line

```
<span>{lifecycleSummary.active} active</span>
<span>{lifecycleSummary.inUse} in use</span>
<span>{lifecycleSummary.deprecated} deprecated</span>
```

{lifecycleSummary.retirementCandidates} retirement candidates

Why

Once families exist, end-of-life risk should be visible at the family level too.

---

11. Row-Level Visual Rules

Lifecycle needs clear but measured scan treatment.

Recommended rules

- `draft` → neutral muted tone
- `active` → neutral/primary-adjacent
- `in_use` → stronger active emphasis
- `degraded` → warning/failure-adjacent
- `deprecated` → warning tone
- `archived` → muted subdued tone
- `retirement candidate` → soft muted callout, not red

Important distinction

Do not style `deprecated` the same as `failed`.
Deprecated is a managed state, not necessarily an incident.

---

12. Deprecation-State Semantics

Treat deprecation as a managed rollout process, not one binary tag.

Good states

- `none`
- `planned`

- `announced`
- `in_progress`
- `completed`

Why this matters

A deprecated assembly might still be:

- live
- in use
- under migration
- awaiting retirement

That nuance is exactly why this page needs a lifecycle layer.

---

13. Retirement Candidate Rules

A retirement candidate should be conservative by default.

Safe first-pass heuristic

An assembly is a retirement candidate when:

- pipeline status is not running
- lifecycle is not active/in use
- usage is not live
- not currently blocked by failure triage
- not still being actively modified

Practical first-pass version

The helper above already gives a workable approximation. Start there, refine later when telemetry is richer.

---

14. Empty-State Copy Extension

Once lifecycle views exist, filtered empty states should be more specific than the current run-oriented wording, which still says “No runs yet” or “No [filter] runs.”

runs

Examples

- `No deprecated assemblies`
- `No retirement candidates`
- `No archived assemblies`
- `No degraded assemblies`

Why

Lifecycle-specific empty states make the saved views feel intentional.

---

15. Acceptance Criteria

Pass 8 is complete when:

- lifecycle is a real second state axis alongside execution status
- lifecycle filter supports active, in use, degraded, deprecated, archived, and retirement candidates
- lifecycle/deprecation logic is centralized in helpers
- overview cards can surface lifecycle posture
- quick-detail drawer includes a lifecycle section
- family headers can summarize lifecycle mix
- the page can support deprecation and retirement workflows, not just active pipeline monitoring

This is the missing long-term management layer the current page cannot express because it is still centered on execution status, status counts, and run-oriented filter language.

---

16. Result of Pass 8

After this pass, the Assemblies page becomes much more than a monitoring surface. It starts functioning as a real lifecycle registry where users can see what is still alive, what is fading out, what is deprecated, and what is ready for retirement.

Pass 9 — Exact API / Live-Usage Telemetry

Pass 9 — Exact API / Live-Usage Telemetry and Ecosystem-Dependency Block

Pass Goal

Add the final systems layer so the Assemblies page can answer:

- is this assembly actually in use?
- how much traffic or activity does it have?
- what depends on it?
- what does it depend on?
- is it isolated, central, or risky?
- is it safe to deprecate or will other assemblies break?

That is the missing bridge between:

- assembly registry
  and
- ecosystem operations

---

1. Why this pass matters

After Passes 1–8, the page can already express:

- identity
- execution state
- lifecycle
- ownership
- grouping
- family context
- quick inspection

But it still cannot express:

- real usage

- consumer activity
- upstream/downstream relationships
- ecosystem blast radius

Without this pass, a user can see that an assembly is “completed” or even “deprecated,” but they still cannot reliably tell whether it is **quiet**, **live**, or **dangerously depended on**.

The current page proves that gap because it only renders:

- project/idea identity
- status
- pipeline
- preset
- elapsed/duration
- updated
- actions, with status filtering derived from assembly status only.

---

2. New data model additions

Add these fields to the assembly page model when available:

```
usageState?: "live" | "warm" | "idle" | "dormant" | "blocked" | "unknown" | null;
requestsLast1h?: number | null;
requestsLast24h?: number | null;
lastActivityAt?: string | null;
activeConsumers?: number | null;
errorRatePct?: number | null;
p95LatencyMs?: number | null;

upstreamDependencies?: Array<{
 id: string;
 name: string;
 type?: string;
 status?: "healthy" | "degraded" | "unknown";
}> | null;

downstreamDependents?: Array<{
 id: string;
 name: string;
 type?: string;
 criticality?: "low" | "medium" | "high";
```

```
}> | null;

ecosystemRole?: "core" | "supporting" | "adapter" | "integration" | "edge" | "experimental" | null;
dependencyRisk?: "low" | "medium" | "high" | "critical" | null;
sharedApis?: string[] | null;
sharedDatasets?: string[] | null;
```

Safe fallback rule

Until real telemetry exists, keep conservative placeholder derivation:

- running → `usageState = "live"`
- completed → `usageState = "idle"`
- queued → `usageState = "unknown"`
- failed → `usageState = "blocked"`

That placeholder is consistent with the page's existing execution-first model, which is already driven from assembly status.

runs

---

3. Central usage helpers

Add these above render so the page can derive usage and ecosystem posture consistently.

Usage-state helper

```
const getUsageState = (assembly: Assembly) => {
 const explicit = (assembly as any).usageState;
 if (explicit) return explicit;

 if (assembly.status === "running") return "live";
 if (assembly.status === "completed") return "idle";
 if (assembly.status === "failed") return "blocked";
 return "unknown";
};
```

Requests helper

```
const getRequests24h = (assembly: Assembly) =>
```

```
 (assembly as any).requestsLast24h ?? null;
```

Last-activity helper

```
const getLastActivityAt = (assembly: Assembly) =>
 (assembly as any).lastActivityAt || null;
```

Consumer-count helper

```
const getActiveConsumers = (assembly: Assembly) =>
 (assembly as any).activeConsumers ?? 0;
```

Dependency helpers

```
const getUpstreamDependencies = (assembly: Assembly) =>
 Array.isArray((assembly as any).upstreamDependencies)
   ? (assembly as any).upstreamDependencies
   : [];

const getDownstreamDependents = (assembly: Assembly) =>
 Array.isArray((assembly as any).downstreamDependents)
   ? (assembly as any).downstreamDependents
   : [];
```

Ecosystem-risk helper

```
const getDependencyRisk = (assembly: Assembly) => {
 const explicit = (assembly as any).dependencyRisk;
 if (explicit) return explicit;

 const downstream = getDownstreamDependents(assembly).length;
 if (downstream >= 10) return "critical";
 if (downstream >= 5) return "high";
 if (downstream >= 2) return "medium";
 return "low";
};
```

---

4. Usage column upgrade

Pass 2 added a `Usage` column conceptually. Pass 9 makes it meaningful.

Exact Usage cell block

```
<td className="px-4 py-3 hidden lg:table-cell">
 <div className="flex flex-col gap-1">
   <span className="text-xs text-[hsl(var(--foreground))] capitalize">
     {getUsageState(a)}
   </span>

   {getRequests24h(a) !== null ? (
     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       {getRequests24h(a)?.toLocaleString()} req / 24h
     </span>
   ) : (
     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       No telemetry yet
     </span>
   )}

   {getLastActivityAt(a) && (
     <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
       Last activity {formatDate(getLastActivityAt(a))}
     </span>
   )}
 </div>
</td>
```

Why this is the right shape

The current page already shows execution timing like elapsed, duration, and updated time, but those are internal run metrics. Usage needs to show **external relevance** instead.

---

5. Add a new Ecosystem column

This is the last major table addition.

New table target

By the end of Pass 9, the table should be:

- Assembly

- Family
- Status
- Lifecycle
- Usage
- Ecosystem
- Pipeline
- Owner
- Updated
- Actions

Exact header addition

```
<th className="text-left px-4 py-2.5 text-system-label hidden xl:table-cell">
 Ecosystem
</th>
```

Exact Ecosystem cell block

```
<td className="px-4 py-3 hidden xl:table-cell">
 <div className="flex flex-col gap-1">
   <span className="text-xs text-[hsl(var(--foreground))] capitalize">
     {(a as any).ecosystemRole || "Unclassified"}
   </span>

   <span className="text-[10px] text-[hsl(var(--muted-foreground))]">
     {getDownstreamDependents(a).length} dependents · {getUpstreamDependencies(a).length}
upstream
   </span>

   <span className="text-[10px] text-[hsl(var(--muted-foreground))] capitalize">
     {getDependencyRisk(a)} risk
   </span>
 </div>
</td>
```

Why

This makes “importance in the system” visible in the table without forcing a graph view.

---

6. Usage filter upgrade

The earlier filter model should now use actual usage helpers instead of placeholder-only logic.

Exact type

```
type UsageFilter = "all" | "live" | "warm" | "idle" | "dormant" | "blocked" | "unknown";
```

Exact matching block

```
const usageState = getUsageState(a);

const matchesUsage =
 usageFilter === "all" ? true : usageState === usageFilter;
```

Add ecosystem-specific filters

Add another compact filter:

```
type EcosystemFilter = "all" | "core" | "supporting" | "adapter" | "integration" | "edge" | "high_risk";
const [ecosystemFilter, setEcosystemFilter] = useState<EcosystemFilter>("all");
const ecosystemRole = (a as any).ecosystemRole || "unclassified";
const dependencyRisk = getDependencyRisk(a);

const matchesEcosystem =
 ecosystemFilter === "all"
   ? true
   : ecosystemFilter === "high_risk"
   ? dependencyRisk === "high" || dependencyRisk === "critical"
   : ecosystemRole === ecosystemFilter;
```

This should join the layered filter pipeline alongside the earlier status/family/lifecycle/owner/risk filters.

---

7. Saved views expansion

Add live-usage and dependency-aware views.

Add these saved views

- `live`
- `idle`

- `high_dependency_risk`
- `orphaned`
- `core_services`

Exact additions

```
type SavedView =
 | "all"
 | "active"
 | "failed"
 | "stalled"
 | "in_use"
 | "unowned"
 | "no_family"
 | "deprecated_candidates"
 | "at_risk"
 | "no_control_plane"
 | "agentless"
 | "partial_assignment"
 | "deprecated"
 | "retirement_candidates"
 | "archived"
 | "degraded"
 | "live"
 | "idle"
 | "high_dependency_risk"
 | "orphaned"
 | "core_services";
```

Exact matching logic additions

```
: savedView === "live"
? getUsageState(a) === "live"
: savedView === "idle"
? getUsageState(a) === "idle"
: savedView === "high_dependency_risk"
? ["high", "critical"].includes(getDependencyRisk(a))
: savedView === "orphaned"
? getUpstreamDependencies(a).length === 0 && getDownstreamDependents(a).length === 0
: savedView === "core_services"
? (a as any).ecosystemRole === "core"
```

Why

These views make the page useful for:

- traffic triage
- dependency cleanup
- system-centrality review
- orphan detection

---

8. Top summary cards expansion

Pass 4 added execution summary cards. Pass 8 added lifecycle cards. Pass 9 should add usage and ecosystem cards.

Add these cards

- Live Usage
- Idle
- High Risk Dependencies
- Orphaned
- Core Services
- Active Consumers

Exact derived metrics

```
const liveUsageCount = assemblies.filter((a) => getUsageState(a) === "live").length;
const idleUsageCount = assemblies.filter((a) => getUsageState(a) === "idle").length;
const highDependencyRiskCount = assemblies.filter((a) =>
 ["high", "critical"].includes(getDependencyRisk(a))
).length;
const orphanedCount = assemblies.filter((a) =>
 getUpstreamDependencies(a).length === 0 && getDownstreamDependents(a).length === 0
).length;
const coreServicesCount = assemblies.filter((a) => (a as any).ecosystemRole ===
"core").length;
const totalActiveConsumers = assemblies.reduce((sum, a) => sum + getActiveConsumers(a),
0);
```

Card click mapping

- Live Usage → `setSavedView("live")`

- Idle → `setSavedView("idle")`
- High Risk Dependencies → `setSavedView("high_dependency_risk")`
- Orphaned → `setSavedView("orphaned")`
- Core Services → `setSavedView("core_services")`

Why

This gives the page a true “what matters right now” layer that goes beyond pipeline execution. The current page’s counts only reflect execution status totals.

---

9. Quick-detail drawer upgrade

The drawer needs a dedicated **Usage & Ecosystem** section.

Exact drawer block

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
  Usage & Ecosystem
 </p>

 <div className="grid grid-cols-2 gap-x-4 gap-y-3 text-sm">
  <div>
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Usage State
   </p>
   <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
    {getUsageState(selectedAssembly)}
   </p>
  </div>

  <div>
   <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
    Requests / 24h
   </p>
   <p className="mt-1 text-[hsl(var(--foreground))]">
    {getRequests24h(selectedAssembly)?.toLocaleString() ?? "Not tracked"}
   </p>
  </div>
```

```
  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Active Consumers
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {getActiveConsumers(selectedAssembly)}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Last Activity
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))]">
      {getLastActivityAt(selectedAssembly)
        ? formatDate(getLastActivityAt(selectedAssembly))
        : "Not tracked"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Ecosystem Role
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
      {(selectedAssembly as any).ecosystemRole || "Unclassified"}
    </p>
  </div>

  <div>
    <p className="text-[10px] uppercase tracking-wide text-[hsl(var(--muted-foreground))]">
      Dependency Risk
    </p>
    <p className="mt-1 text-[hsl(var(--foreground))] capitalize">
      {getDependencyRisk(selectedAssembly)}
    </p>
  </div>
 </div>
</div>
```

---

10. Add dependency sections in drawer

The drawer should also expose actual relationships.

Upstream section

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
   Upstream Dependencies
 </p>

 <div className="flex flex-wrap gap-2">
   {getUpstreamDependencies(selectedAssembly).length > 0 ? (
     getUpstreamDependencies(selectedAssembly).map((dep: any) => (
       <span
         key={dep.id}
         className="px-2 py-1 rounded-full text-[11px] border border-[hsl(var(--border))]
text-[hsl(var(--foreground))]"
       >
         {dep.name}
         {dep.type ? ` · ${dep.type}` : ""}
       </span>
     ))
   ) : (
     <span className="text-xs text-[hsl(var(--muted-foreground))]">
       No upstream dependencies
     </span>
   )}
 </div>
</div>
```

Downstream section

```
<div className="px-5 py-4 border-b border-[hsl(var(--border))]">
 <p className="text-[11px] uppercase tracking-wide text-[hsl(var(--muted-foreground))] mb-3">
   Downstream Dependents
 </p>

 <div className="flex flex-wrap gap-2">
   {getDownstreamDependents(selectedAssembly).length > 0 ? (
     getDownstreamDependents(selectedAssembly).map((dep: any) => (
       <span
         key={dep.id}
         className="px-2 py-1 rounded-full text-[11px] border border-[hsl(var(--border))]
text-[hsl(var(--foreground))]"
       >
```

```
        {dep.name}
        {dep.criticality ? ` · ${dep.criticality}` : ""}
      </span>
    ))
  ) : (
    <span className="text-xs text-[hsl(var(--muted-foreground))]">
      No downstream dependents
    </span>
  )}
 </div>
</div>
```

Why

This is the minimum viable topology layer without needing a full graph UI.

---

11. Family header rollups

In Family View, family headers should now summarize not just execution/lifecycle/responsibility, but ecosystem importance.

Exact helper

```
const getFamilyEcosystemSummary = (assemblies: Assembly[]) => {
 const live = assemblies.filter((a) => getUsageState(a) === "live").length;
 const highRisk = assemblies.filter((a) =>
   ["high", "critical"].includes(getDependencyRisk(a))
 ).length;
 const totalDependents = assemblies.reduce(
   (sum, a) => sum + getDownstreamDependents(a).length,
   0
 );
 const totalConsumers = assemblies.reduce(
   (sum, a) => sum + getActiveConsumers(a),
   0
 );

 return {
   live,
   highRisk,
```

```
  totalDependents,
  totalConsumers,
 };
};
```

Add to family summary line

```
<span>{ecosystem.live} live</span>
<span>{ecosystem.highRisk} high-risk</span>
<span>{ecosystem.totalDependents} dependents</span>
<span>{ecosystem.totalConsumers} consumers</span>
```

Why

This gives you family-level ecosystem posture without needing a separate page.

---

12. Orphan and criticality semantics

This pass should introduce two useful classifications:

Orphaned assembly

No upstream dependencies and no downstream dependents.

Use case:

- cleanup
- classification
- archive review
- possible dead asset

Critical assembly

High downstream dependency count or explicitly high dependency risk.

Use case:

- change control
- deprecation caution
- incident awareness

- priority hardening

These concepts should appear in:

- saved views
- cards
- drawer
- future badges

---

13. Row-level visual rules

Usage and dependency context should influence scan quality, but not overwhelm it.

Recommended emphasis

- `live` usage → stronger foreground emphasis
- `idle` / `dormant` → subdued
- `blocked` → warning/failure-adjacent
- high dependency risk → subtle warning tag
- orphaned → muted neutral tag
- core role → neutral/high-value tag

Important

Do not style “live” as failure/alert.
Usage intensity is importance, not incident.

---

14. Backend/API shape recommendation

The current page only queries `/api/assemblies` and deletes assemblies via `/api/assemblies/:id`, while active-run context comes from `usePipelineStatus(hasActiveRuns)`.

runs

For this pass, the cleanest direction is:

Option A — enrich `/api/assemblies`

Return telemetry/dependency summary fields directly inside each assembly payload.

Best for:

- simpler page integration
- one request model
- easier table rendering

Option B — add a sidecar summary endpoint

Example:

- `GET /api/assemblies/telemetry-summary`
- `GET /api/assemblies/dependency-summary`

Best for:

- heavier or separately refreshed telemetry
- lower coupling with core assembly records

Recommendation

Use **summary fields on `/api/assemblies` first**, then split later if the payload becomes too heavy.

---

15. Acceptance criteria

Pass 9 is complete when:

- usage is a real page concept, not a placeholder label
- assemblies can show requests/activity/consumer hints
- the table exposes ecosystem importance through an Ecosystem column
- saved views can identify live, idle, orphaned, and high-risk assemblies
- quick-detail drawer shows usage and dependency sections
- family headers can summarize ecosystem posture
- the page can support deprecation and change decisions with actual dependency context

This is the missing layer the current page cannot provide because it is still built around assembly status, counts, stage rails, and row actions without any dependency or live-usage dimension.

runs

---

16. Final planned-pass outcome

Once Pass 9 is in place, the Assemblies page will cover the full management stack:

- identity
- execution
- filtering
- overview
- quick inspection
- family grouping
- responsibility
- lifecycle
- usage and ecosystem dependency

That completes the main planned overhaul from **Runs** into **Assemblies**.

Final Cleanup / Rollout Pass

Final Cleanup / Rollout Pass

This is the wrap-up pass after the 9 major passes. It is not a new feature layer. It is the pass that makes the overhaul shippable.

Pass 10 — cleanup, integration, and rollout

Goal

Turn the Assemblies overhaul from a set of good page changes into a stable release plan.

1. Naming cleanup

Make sure all visible page language is consistent:

- Runs → Assemblies
- New Run → New Assembly
- No runs yet → No assemblies yet
- run/workbench wording removed where the object is really the assembly

2. Route and nav cleanup

Lock in:

- nav label = Assemblies
- page component name = AssembliesPage
- preferred route = `/assemblies`
- redirect `/runs` → `/assemblies` if needed
- keep detail route `/assembly/:id`

3. Type cleanup

Create one normalized page-facing type instead of repeated `(a as any)` access.

Recommended shape:

- base assembly fields
- family fields
- lifecycle fields
- ownership fields
- usage fields
- ecosystem/dependency fields

This is one of the biggest cleanup items because the earlier passes used progressive placeholders.

4. Helper consolidation

Move page logic into reusable helpers/selectors:

- `getLifecycleState`
- `getDeprecationState`
- `isRetirementCandidate`
- `getUsageState`
- `getDependencyRisk`
- `getAssignmentHealth`
- `getFamilyHealth`
- `getFamilySummary`
- `getFamilyLifecycleSummary`

- `getFamilyResponsibilitySummary`
- `getFamilyEcosystemSummary`

These should not stay scattered in the component long-term.

5. UI extraction

Break the page into subcomponents.

Recommended extraction:

- `AssembliesHeader`
- `AssembliesOverviewCards`
- `AssembliesStatusChips`
- `AssembliesSavedViews`
- `AssembliesFilterBar`
- `AssembliesTable`
- `AssembliesFamilyHeaderRow`
- `AssembliesQuickDrawer`

That prevents the page from turning into one huge file.

6. Data-loading strategy cleanup

Decide what stays in `/api/assemblies` and what becomes summary-sidecar data.

Best release order:

- keep table-critical summary fields in `/api/assemblies`
- move heavy telemetry later only if needed

7. Empty-state polish

Make empty states context-aware:

- no assemblies at all
- no assemblies for this saved view
- no family matches
- no deprecated assemblies
- no retirement candidates
- no live assemblies

8. Responsive cleanup

Verify final breakpoint behavior:

Mobile:

- Assembly
- Status
- Actions

Medium:

- Family
- Pipeline

Large:

- Lifecycle
- Usage
- Updated

XL:

- Owner
- Ecosystem

Also confirm drawer works well on smaller screens.

9. Interaction cleanup

Lock the page interaction model:

- row click = quick drawer
- explicit open button = full assembly page
- card click = filter/view shortcut
- family header click = collapse/expand group

This should be consistent everywhere.

10. Visual hierarchy cleanup

Make sure scan priority is correct:

1. Assembly identity
2. Status / lifecycle / usage
3. Family / ecosystem / owner
4. Pipeline detail
5. timing and actions

That keeps the page readable.

11. Warning-state calibration

Differentiate clearly between:

- execution failure
- degraded lifecycle
- deprecated managed state
- governance gap
- dependency risk

These should not all look the same.

12. Default view decision

Recommended default:

- Flat View on first load
- Family View available as toggle
- savedView = all
- statusFilter = all

That gives the broadest first scan while still enabling grouped management.

13. Release sequence

Best rollout order:

Release 1

- rename + header rewrite
- table restructure
- filters + saved views
- overview cards

Release 2

- quick drawer
- family grouping
- ownership layer

Release 3

- lifecycle/deprecation
- usage/dependency ecosystem layer

This reduces risk and makes testing easier.

14. QA checklist

Before shipping, verify:

- rename is complete everywhere
- all filters combine correctly
- saved views do not conflict with filters
- grouped and flat view both work
- drawer opens/closes cleanly
- row actions do not misfire
- empty states are correct
- counts/cards match filtered data where expected
- no broken behavior when optional fields are missing
- no layout break on narrow widths

15. Success definition

The update is done when the page no longer feels like a renamed run log and instead clearly behaves as:

- an assemblies registry
- an operations surface
- a lifecycle manager
- a responsibility map
- an ecosystem awareness page

Final pass count

That brings the full plan to:

1. UI rewrite
2. Table expansion
3. Filters + saved views
4. Overview cards
5. Quick-detail drawer
6. Family grouping
7. Ownership / control plane / agents
8. Lifecycle / deprecation / retirement
9. Usage telemetry / ecosystem dependency
10. Cleanup / rollout

So the full overhaul is **10 passes total**, with **9 major feature passes** and **1 final cleanup pass**.